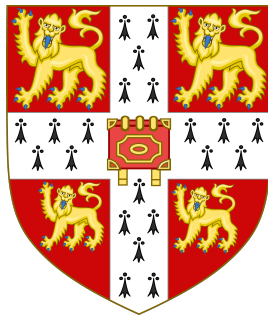


Model Merging and Task Overfitting in Speech LLMs



Ebinezer Rajaram

Trinity College
Department of Engineering
University of Cambridge

Supervisors: Professor Phil Woodland and Dr Guangzhi Sun

This dissertation is submitted for the degree of

Master of Engineering

1st June 2026

Technical Abstract

Model Merging and Task Overfitting in Speech LLMs

Ebinezer Rajaram, Trinity College, University of Cambridge

Modern speech processing spans diverse behaviours, from transcription and spoken-language understanding to speaker and acoustic-event recognition. Speech large language models (Speech LLMs) extend text-based language models with audio conditioning, so that a single shared decoder handles all tasks: it takes speech and a task-specific prompt as input and produces a text response. In principle, one model can serve many tasks, but adapting it to do so is costly, as joint adaptation requires pooled multi-task training while training a separate full model for each task wastes parameters. Low-rank adaptation (LoRA) provides a storage-efficient alternative: each task-specific adaptation is represented as a compact weight update applied to a shared frozen backbone. These adapters can be trained, stored, and updated independently, giving useful storage modularity. Behavioural modularity is harder to guarantee, since applying one adapter may alter performance on tasks it was never trained for. This project studies whether independently trained speech-task adapters can be merged after training by calibrating how much each adapter contributes, and compares calibrated merging with joint multi-task learning.

The experiments use Qwen2.5-Omni-3B, a frozen 3-billion-parameter Speech LLM backbone, with a shared LoRA configuration across all adapters. Seven varied speech tasks are evaluated: automatic speech recognition (ASR), emotion recognition (ER), intent classification (IC), keyword spotting (KWS), language identification (LID), speaker verification (SV), and vocal sound classification (VS). These tasks differ in acoustic evidence, output format, and evaluation metric, ranging from free-form transcription to short-label classification and binary verification. Each task is adapted independently, producing a fixed weight update, or task vector, relative to the frozen backbone. Results are reported using both raw task metrics and headroom-normalised recovery, defined as the fraction of a single-task adapter’s improvement over the frozen backbone that is preserved when adapters are merged. On this scale, 0 corresponds to the frozen backbone and 1.0 to the single-task reference.

Cross-task evaluation reveals that applying one task’s adapter changes performance on tasks it was never trained for. The interference is directional and asymmetric: the IC and KWS adapters severely reduce LID accuracy, the VS adapter reduces IC accuracy partly through output-format mismatch, and several non-ASR adapters unexpectedly lower ASR word error rate. These failures show that an adapter can perform well on its own held-out task while degrading other tasks that share the same backbone. This project calls this harmful subcase task overfitting. Unlike ordinary train–test overfitting, it is a shift in shared decoder behaviour caused by fine-tuning towards one task’s prompt and response format. Single-task evaluation cannot detect this failure mode, so it must be measured explicitly on the adapter set being deployed, motivating calibrated merging.

Naive merging performs poorly under these interference patterns. Equal-weight averaging across all seven task vectors achieves a mean recovery of only 0.551, far below full recovery of the single-task gains, and conflict-aware filtering methods such as TIES and DARE improve some tasks but leave worst-task recovery no higher than 0.424. The principal failure is poor calibration of the combined update magnitude, not fundamental adapter incompatibility: task-vector directions are informative, but their summed magnitude is badly scaled. Selecting a single global scalar coefficient on the development set already raises mean recovery to 0.956; replacing that scalar with supervised layer-wise coefficients brings it to 1.011, with worst-task recovery at 0.773 on emotion recognition. This layer-wise calibration is the strongest static merge approach and the more reliable choice when worst-task recovery is the key concern. Emotion recognition remains the hardest task to recover across calibrated methods, making it the main remaining bottleneck.

A matched multi-task learning (MTL) baseline tests whether calibrated merging can compete with joint retraining on the same task suite. In this fixed-suite setting, MTL achieves the strongest non-ASR mean recovery (0.997), consistent with the advantage of optimising shared LoRA parameters directly on pooled task data. However, MTL gives higher ASR word error rate (2.24%) than global scalar calibration (1.72%) or layer-wise calibration (1.81%). Calibrated merging is more practical when adapters already exist, the task set may change, or source data cannot be pooled, and it is also preferable when ASR word error rate or worst-task recovery is the main concern. When the task suite is fixed, source data can be pooled, and non-ASR mean recovery is the priority, MTL is the better-supported choice.

The changing-task setting tests how well calibrated merging adapts as the task suite grows. Across five diagnostic task-addition paths, all methods are order-dependent: the same tasks show different acquisition and retention behaviour depending on when they are added. In this setting, layer-wise continual merging gives the best overall acquisition–retention balance when the goal is to reuse fixed adapters, because it can recalibrate the incoming adapter against the previous merge at each layer. Sequential MTL is also strong when continued retraining on accumulated task data is allowed, often giving the highest new-task acquisition, but with retained-task failures consistent with catastrophic forgetting. A simpler two-coefficient merge is competitive in some cases, but less reliable on the weakest retained task. Continual merging cannot be judged by mean recovery alone; both the incoming task and the weakest retained task must be monitored.

The central conclusion is that independently trained adapters are useful reusable updates, but they are not plug-and-play behavioural modules: their behaviour does not remain isolated simply because they are trained and stored separately. Storage modularity alone is not enough: modular reuse requires explicit cross-task measurement, development-set coefficient calibration, and worst-task monitoring. With these controls, calibrated merging turns fixed speech-task adapters into a practical reuse mechanism. Existing adapters can then be merged again as requirements change, reducing the need for repeated joint retraining while keeping worst-task recovery visible at each merge step.

Contents

Technical Abstract	i
1 Introduction	1
1.1 Modular Adaptation for Speech LLMs	1
1.2 Cross-Task Interference and Task Overfitting	1
1.3 Calibrated Post-Hoc Merging	2
1.4 Aim, Research Questions, and Contributions	3
2 Background and Related Work	4
2.1 Speech LLMs and the Shared Decoder	4
2.2 Parameter-Efficient Fine-Tuning and LoRA	6
2.3 Speech Tasks and Cross-Task Interference	6
2.4 Model Merging and Task-Vector Composition	7
2.5 Multi-Task Learning and Negative Transfer	9
2.6 Continual Adapter Merging	9
2.7 Research Gap	10
3 Methodology	11
3.1 LoRA Task Adapters and Task Vectors	11
3.2 Post-Hoc Adapter Merging	12
3.3 Recovery and Interference Metrics	13
3.4 Matched MTL Baseline	14
3.5 Continual Modular Composition	15
4 Experimental Design	16
4.1 Backbone, Tasks, and Evaluation Metrics	16
4.2 Prompting, Decoding, and Output Parsing	17
4.3 Training and Capacity Controls	17
4.4 Merging and Matched MTL Protocols	18
4.5 Continual Protocol	19
4.6 Reproducibility Scope	20
5 Cross-Task Adapter Interference	21
5.1 Single-Task and Cross-Task Adapter Behaviour	21
5.2 Asymmetry and Parsing Diagnostics	22
5.3 Weight-Space Diagnostics	24
5.4 Implications for Merging	24

6	Static Model Merging	25
6.1	Calibration Method Selection	25
6.2	Seven-Task Simultaneous Merge	26
6.2.1	Layer-wise Coefficient Structure	28
6.2.2	ASR Raw WER and Recovery	28
6.3	Leave-One-Out Adapter Coverage	29
7	Fixed-Suite MTL Comparison	31
7.1	Absolute Task Performance	31
7.2	Recovery and Balance	32
7.3	Held-Out Diagnostic Checks	32
7.3.1	SQA Format Stability	32
7.3.2	Harder-Split ASR Check	33
7.4	Six-Task Sensitivity	33
7.5	Practical Interpretation	34
8	Continual Adapter Merging	35
8.1	Setup and Success Criteria	35
8.2	ASR-Held-Out Path	36
8.3	Acoustic-First Path	36
8.4	Acquisition–Retention Balance Across Five Paths	37
9	Discussion and Conclusions	38
	Bibliography	40
A	Supplementary Diagnostics	42
A.1	Output-Format Validity	42
A.2	Held-Out SQA Diagnostic	42
A.3	LID Confusion Diagnostics	43
A.4	ASR Same-Sample WER Diagnostic	44
A.5	Continual Path Sweep	44
B	Risk Assessment Retrospective	46

Chapter 1

Introduction

1.1 Modular Adaptation for Speech LLMs

Recent audio-language models connect a speech encoder to a large language model decoder, so that the same text-generation mechanism handles transcription, spoken-language understanding, speaker recognition, and acoustic-event recognition within a single deployable system [1, 2, 3, 4, 5]. In this project, a Speech LLM is an audio-conditioned generative model that receives speech and a task prompt, then produces the answer as text. Because all tasks produce text through the same decoder, adapting it for one task can shift the output behaviour it produces for others. Therefore, modular adaptation demands more than single-task performance: independently trained task adapters must also be reusable without disrupting other behaviours.

Low-rank adaptation (LoRA) offers a storage-efficient route [6]. By freezing the shared backbone and storing task-specific updates as compact trainable modules, LoRA allows adapters to be developed independently while the underlying Speech LLM remains unchanged. In the backbone studied here, some adapters refine well-established model behaviours, while others specialise weaker or more format-specific ones. These updates still act through the shared decoder, so reliable adapter reuse involves preservation as well as acquisition: an adapter should improve its own task without disrupting behaviours the backbone or other adapters already support. LoRA provides storage modularity, but the more demanding requirement is behavioural modularity, since activating or merging an adapter should not cause harmful off-task changes.

Joint multi-task learning (MTL) addresses multi-task adaptation by pooling task data and jointly training a shared LoRA configuration. This works well when the full task set is fixed and joint retraining is feasible, but it requires pooled task data, a defined task mixture, and a fresh training run whenever the suite changes. These conditions are restrictive when task adapters are developed under different data agreements, at different times, or updated independently. The central question is whether independently trained adapters can be assembled into a reliable multi-task system through calibrated post-hoc merging, without joint retraining.

1.2 Cross-Task Interference and Task Overfitting

Cross-task interference is any behavioural change produced when an adapter is evaluated on a task it was not trained for. It is task-pair-specific and directional: adapter A may

affect task B without adapter B producing the same effect on task A . Not all such interference is harmful. Some adapters are broadly preserving, while others improve tasks they were not trained for. This mixture of harmful and beneficial off-task effects makes adapter composition an empirical calibration problem rather than a question of whether adapters are simply compatible or incompatible.

In this project, task overfitting denotes the harmful subcase: an adapter improves its own held-out task while degrading one or more other tasks supported by the same backbone. It is distinct from ordinary train–test overfitting, which concerns generalisation within a task, and from negative transfer during joint MTL, which arises during shared-parameter training. Task overfitting can occur even when the adapter performs well on its source task, because specialising the shared decoder towards one prompt and output format can reshape behaviour that other tasks also depend on. Chapter 5 measures both positive and negative cross-task interference across all ordered source-adapter and evaluation-task pairs.

A clean modular view, in which each adapter adds one task behaviour while leaving others unchanged, is too optimistic for the task suite studied here. The seven tasks are automatic speech recognition (ASR), which transcribes spoken audio; emotion recognition (ER), which classifies expressed emotion; intent classification (IC), which identifies utterance intent; keyword spotting (KWS), which recognises spoken keyword classes; language identification (LID), which determines the spoken language; speaker verification (SV), which determines whether two audio clips come from the same speaker; and vocal sound classification (VS), which labels non-verbal vocal events. All seven tasks produce text through the same autoregressive decoder, but draw on different acoustic, lexical, semantic, speaker, and paralinguistic evidence and impose different output formats. Therefore, their mutual cross-task effects are an empirical question, not a guarantee.

1.3 Calibrated Post-Hoc Merging

Because cross-task interference is structured rather than uniformly destructive, some merge failures may be corrected by choosing better composition coefficients rather than by retraining the shared parameters. Model merging provides the post-hoc mechanism: each task adapter is trained separately, then its effective LoRA update is combined arithmetically after training. Relevant operations include uniform averaging, scalar task arithmetic, sparsity- and sign-based filtering, and supervised coefficient assignment [7, 8, 9, 10, 11]. The calibrated methods evaluated here keep those adapters fixed after training and use development data only to select their composition coefficients.

When trained adapters already exist, changing the deployed task subset requires only coefficient recalibration rather than source-adapter or shared-parameter retraining, and a new coefficient set can be derived whenever task requirements change. If each task’s training data is held or governed separately, the composition step requires only labelled development examples for the target task set, without needing to pool the underlying source data.

MTL has a clear information advantage when all task data are jointly available, because it optimises the shared LoRA parameters directly against the full task mixture. Post-hoc merging solves a different optimisation problem: it selects coefficients over fixed task vectors, not the underlying adapter weights. Therefore, the choice depends on deployment context. Joint MTL favours a fixed, jointly pooled task suite; calibrated merging is more relevant when source adapters already exist, the deployment task set changes, source data are governed separately, or worst-task performance must be protected.

1.4 Aim, Research Questions, and Contributions

The aim is to determine whether independently trained LoRA adapters for diverse speech tasks are behaviourally modular, and whether calibrated post-hoc merging can assemble them into a reliable multi-task system without joint retraining.

The project is organised around four research questions (RQs):

1. **RQ1.** How behaviourally modular are independently trained speech-task LoRA adapters under a shared Speech LLM backbone (Chapter 5)?
2. **RQ2.** Can calibrated post-hoc merging recover balanced multi-task performance from those adapters (Chapter 6)?
3. **RQ3.** Which calibration mechanisms matter most: global-scale calibration, sparsity and conflict filtering, or supervised layer-wise coefficients (Chapter 6)?
4. **RQ4.** How does calibrated merging compare with matched MTL in fixed-suite settings and with continual task-addition baselines in changing-task settings (Chapters 7–8)?

The project makes four principal contributions. First, direct cross-task measurement shows that task overfitting is invisible to single-task evaluation: adapters are not behaviourally modular, and interference is structured and asymmetric. Second, post-hoc LoRA merging experiments show that global-scale calibration corrects much of naive averaging, while supervised layer-wise calibration gives the strongest worst-task recovery among static methods. Third, comparisons with matched MTL and continual task-addition baselines identify when fixed-adapter reuse is viable and when joint retraining has an advantage. Fourth, the seven-task Speech LLM protocol enables these comparisons through raw task metrics and headroom-normalised recovery.

The following chapters develop this argument. Chapter 2 provides background on Speech LLM adaptation, LoRA, model merging, MTL, and continual learning. Chapters 3–4 define the formal metrics and experimental protocol. Chapter 5 measures cross-task interference in the source adapters; Chapter 6 tests whether calibrated static merging can correct it; Chapter 7 compares the primary calibrated merge methods with matched MTL; Chapter 8 evaluates continual task-addition; and Chapter 9 synthesises the findings.

Chapter 2

Background and Related Work

This chapter develops four ideas that underlie the project’s methodology and results. Together, these ideas explain why independently trained adapters do not automatically merge without calibration. All speech tasks in this project share a single autoregressive decoder, so a task-specific adapter can change the output behaviour the backbone produces for other tasks. LoRA provides compact, independently stored updates for each task. Model merging provides a post-hoc route to multi-task behaviour when merging is calibrated. The chapter introduces multi-task learning as the matched training baseline and situates continual adapter merging as the setting where the task set changes over time.

2.1 Speech LLMs and the Shared Decoder

A language model decoder is a neural network trained to predict the next token given a preceding sequence. Tokens are discrete units from a fixed vocabulary, such as words, subwords, or characters, over which the model defines a probability distribution. At inference time, the model generates text autoregressively: it produces one token at a time, conditioning each prediction on the input and on all tokens already generated. The input provided to the model is the prompt, and the sequence produced in response is the generated output. The same model parameters can support different tasks by changing the prompt, without changing the weights.

A Speech LLM is an audio-conditioned generative model: it receives speech or audio together with a text prompt, and produces a text response through an autoregressive decoder. In a text-only LLM, the model assigns probability

$$p(y_{1:T} \mid x_{1:N}) = \prod_{t=1}^T p(y_t \mid y_{1:t-1}, x_{1:N}), \quad (2.1)$$

where $x_{1:N}$ is the text context and $y_{1:T}$ is the generated response. For a Speech LLM, the corresponding factorisation is

$$p(y_{1:T} \mid a, q) = \prod_{t=1}^T p(y_t \mid y_{1:t-1}, q, f_\phi(a)), \quad (2.2)$$

where a is the audio input, q is the task prompt, and $f_\phi(a)$ is the encoded audio. Each token is generated by conditioning on the encoded audio, the task prompt, and the tokens already generated.

Audio enters the decoder through a two-stage process. An audio encoder first compresses the waveform into a sequence of fixed-dimensional representations. A learnable projector then maps these representations into the same embedding space as text tokens, so the decoder processes a unified sequence of projected audio tokens, instruction tokens, and generated response tokens. Recent Speech LLMs differ in their encoder architectures and connectors, but share this basic structure of mapping audio into a language-model generation pathway [1, 2, 3, 4, 5].

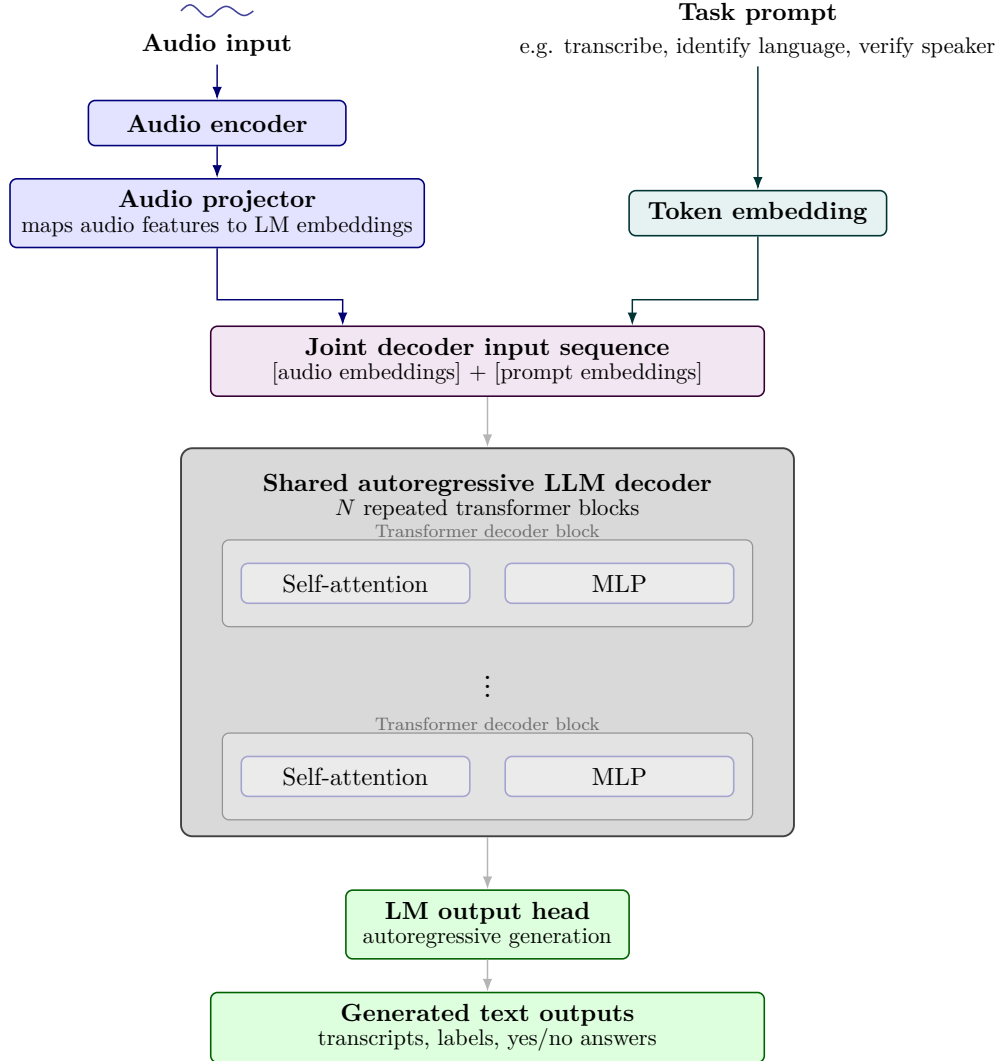


Figure 2.1: Conceptual Speech LLM architecture. Audio is encoded into continuous representations and projected into the language-model embedding space alongside task-prompt embeddings. All tasks produce text through the same autoregressive decoder, so a task adapter can affect the outputs that decoder produces for other tasks.

Under this shared decoder, ASR transcripts, emotion labels, intent labels, language labels, speaker-verification answers, and vocal sound labels are all produced through the same pathway. The task is specified by the prompt and expected response format; the underlying text generator is shared across all tasks.

The transformer decoder contains repeated blocks that mix and transform these representations. Self-attention mixes information across context positions, so a generated token

can depend on the audio representation, the instruction, and the tokens generated so far. Feed-forward layers then transform each position’s representation independently [12]. Attention, MLP, and audio projector components are the module types targeted by LoRA updates, as defined in the next section.

This architectural sharing is the reason adapter reuse is not automatically behaviourally modular. The next section defines the LoRA mechanism; Section 2.3 then explains why speech tasks can still interfere once adapters are active.

2.2 Parameter-Efficient Fine-Tuning and LoRA

Parameter-efficient fine-tuning (PEFT) adapts a large pretrained model by training a small set of additional parameters while keeping most backbone weights frozen [13]. A compact task-specific update can then be stored beside a shared model, giving storage modularity, though this does not extend to inference-time behaviour: once activated, the update can still affect other tasks through the shared decoder.

LoRA represents task-specific adaptation as an additive low-rank update [6]. For a frozen weight matrix $W \in \mathbb{R}^{d \times d'}$, the task update is

$$\Delta W = \frac{\alpha}{r} BA, \tag{2.3}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times d'}$ are trainable factor matrices, $r \ll \min(d, d')$ is the low rank, and α scales the update. The adapter therefore adds $r(d + d')$ trainable parameters rather than the dd' parameters required for a full update to W . During the forward pass, the model uses $W + \Delta W$, while the backbone weight W remains fixed. This additive form is what makes LoRA convenient for post-hoc merging: each adapter defines a weight-space update relative to the same frozen backbone.

What matters for merging is the effective LoRA update $\Delta W = (\alpha/r)BA$, not the separately stored factors A and B . Averaging the factors separately is not generally equivalent to averaging the effective updates, since the merged product depends on interactions between A and B . Throughout this project, “task vector” means the collection of effective LoRA updates produced by one source adapter. Chapter 3 formalises this notation and defines the merge parameterisations used in the experiments.

Separate adapters avoid one source of persistent forgetting: training a new task does not overwrite stored adapters for earlier tasks. Whether task vectors are compatible at inference time depends on the tasks they encode; the next section defines the speech task suite and the interference behaviour measured in the experiments.

2.3 Speech Tasks and Cross-Task Interference

The task suite evaluated here is deliberately diverse. The seven tasks are automatic speech recognition (ASR), emotion recognition (ER), intent classification (IC), keyword spotting (KWS), language identification (LID), speaker verification (SV), and vocal sound

classification (VS). Each task is prompted and produces text output through the same autoregressive decoder, but the tasks rely on different acoustic, lexical, semantic, and paralinguistic cues and produce different output forms. ASR uses lexical content and in general generates multi-word transcripts [14]. IC and KWS are label-prediction tasks tied to utterance content [15, 16]. ER and VS depend on acoustic and paralinguistic evidence [17, 18]. SV depends on speaker identity, while LID depends on language-specific phonetic and lexical cues [19, 20]. A shared backbone can support this range through prompting, as demonstrated by recent Speech LLMs [1, 2, 3]. The question here is narrower: whether independently trained LoRA adapters for these tasks remain compatible after training.

These differences make adapter compatibility empirical rather than automatic. An adapter trained for transcription may shift the model towards longer free-form text; one trained for speaker verification may favour binary yes/no responses; and one trained for label prediction may alter the label distribution among short generated answers. Since all of these behaviours pass through the same shared decoder, an off-task change can appear as an incorrect label, a changed transcript distribution, or an invalid response format.

Cross-task interference is any off-task change caused by evaluating an adapter on a task it was not trained for; task overfitting, as defined in Chapter 1, is the harmful subcase where at least one other task’s performance is degraded. Both differ from ordinary train–test overfitting, which concerns generalisation within one task, and from classical catastrophic forgetting, which concerns persistent overwriting during sequential parameter updates [21]. Here, the backbone and stored adapters may remain unchanged, while interference appears because an active adapter changes the shared computation used at inference time.

This distinction matters for modular reuse. Storage modularity means that separate adapters can be kept beside a frozen backbone. Behavioural modularity is stronger: it means that a stored adapter can be reused without harmful off-task changes. Chapter 5 measures this property directly by evaluating each source adapter on every task in the suite. The model-merging literature then provides the merging formalisms used to test whether these task vectors are compatible enough for multi-task reuse.

2.4 Model Merging and Task-Vector Composition

Model merging studies when independently trained weight updates can be combined without retraining. In the setting most relevant here, each update is anchored to a shared initialisation, so fine-tuning can be treated as a vector around the same base model. Yang et al. [22] provide a recent survey. This speech-task setting is a stricter test than many same-domain merge settings, because the source task vectors encode different evidence types, decoding behaviours, and expected output forms.

A basic form of merging is weight averaging. Mode-connectivity work shows that independently trained models can sometimes be linked by paths through weight space along which training loss remains low [23], while model soups show that uniform or validation-guided averaging of fine-tuned checkpoints can improve generalisation [7]. Both rely on updates occupying compatible regions of weight space, an assumption that weakens when tasks

differ in both input evidence and output form.

Task arithmetic makes the shared-initialisation view explicit. Given a pretrained model θ_0 and a fine-tuned checkpoint θ_t , the task vector is $\tau_t = \theta_t - \theta_0$. Because every task vector is measured from the same θ_0 , linear combinations have a natural interpretation: a weighted sum of task vectors blends source contributions around the shared base. Scaling, adding, or subtracting such vectors can steer model behaviour when the vectors do not strongly conflict [8]. For LoRA adaptation, the analogous task vector is the collection of effective updates $(\alpha/r)BA$. The question for this project is whether these speech-task vectors combine approximately linearly around a common frozen Speech LLM, despite the diverse evidence types and output requirements described above. Figure 2.2 illustrates this geometric view using three representative task vectors from the project’s suite.

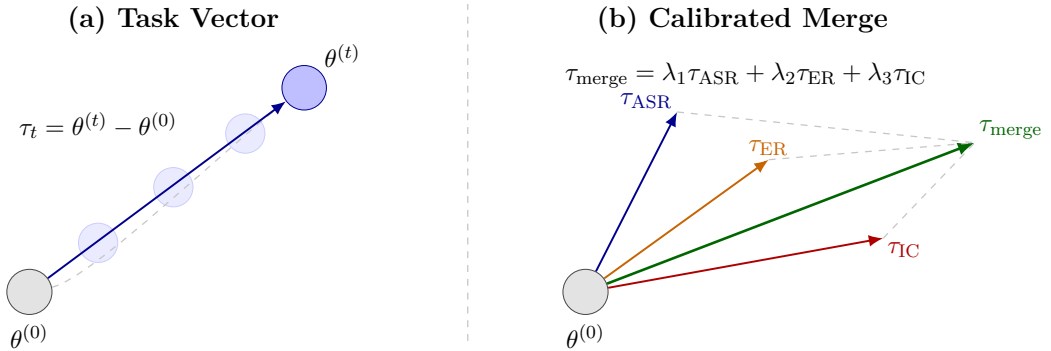


Figure 2.2: Task-vector view of post-hoc Speech LLM adaptation. Each independently trained LoRA adapter defines a task vector τ_t relative to the frozen backbone θ_0 . Post-hoc merging forms a weighted combination of these fixed source vectors, producing a composed model θ_{merge} .

Conflict-aware methods try to repair coordinate-level incompatibilities before or during merging. TIES-Merging trims small changes, resolves sign conflicts by majority sign, and merges retained coordinates [10]. DARE (Drop And REscale) randomly drops and rescales task-vector entries [11]. These methods test whether sparsity, sign agreement, or magnitude filtering can reduce interference. In this project they serve as secondary baselines; the key question is whether observed failures arise from coordinate-level conflicts or from layer-specific, task-pair-specific, or output-format effects.

Coefficient-optimised methods keep source updates fixed while selecting contribution weights using labelled feedback. AdaMerging [24], SuperMerge [25], LoraHub [26], and PEFT arithmetic [9] are representative examples. This project uses coefficient calibration over fixed Speech LLM LoRA task vectors. The primary setting is supervised post-hoc merging: labelled development data are used to choose scalar or layer-wise coefficients, while final claims are reported on held-out test data.

Methods such as AdapterFusion [27] and mixture routing retain multiple adapters and select or blend them at inference time, conditioned on the input or task identity. This is distinct from the fixed-coefficient merging evaluated here, which produces a single composed model without runtime adapter selection and without same-rank recompression of the merged update.

Other methods require information beyond the fixed-coefficient setting evaluated here.

Fisher merging [28], RegMean [29], and representation surgery [30], which aligns internal representations before merging, rely on extra statistics or alignment signals, while task-parameter subspace alignment [31] and SVD-based coordinate alignment of LoRA updates [32] introduce pre-merge transformations. These are natural extensions, but they do not match the project setting: fixed stored LoRA task vectors plus labelled development data for coefficient calibration. Post-hoc merging and joint training are both supervised routes to multi-task performance. The next section defines how they differ in what they optimise.

2.5 Multi-Task Learning and Negative Transfer

Joint multi-task learning (MTL) is the matched training baseline for post-hoc merging. A shared model trained on several tasks can learn representations that are useful across tasks [33, 34]. Its performance is still shaped by task compatibility, pooled data, sampling policy, loss weighting, gradient conflict, and capacity [35]. Accordingly, MTL is a fair baseline rather than a guaranteed upper bound.

The distinction between MTL and calibrated merging is the optimisation variable. MTL trains shared LoRA parameters using pooled task data. Calibrated merging keeps independently trained task vectors fixed and chooses coefficients over them. Both are supervised optimisation settings, but they require different data access and solve different optimisation problems. MTL is appropriate when the task suite is fixed and pooled training is acceptable. Coefficient calibration is more relevant when source adapters already exist, source data cannot be pooled, or the deployment task set changes.

Negative transfer in MTL is distinct from task overfitting in post-hoc merging. In MTL, the failure arises during shared-parameter optimisation when gradients from one task harm another. Task overfitting in this project occurs at inference time: an adapter trained on one task changes shared decoder behaviour in ways that degrade a different task’s evaluation, with no change to any stored weights. Gradient-balancing methods such as PCGrad [36], GradNorm [37], and uncertainty-weighted objectives [38] address the training-time failure but define a separately optimised MTL family. In this project, the matched MTL baseline compares pooled-data shared-adapter training with coefficient calibration over fixed task vectors under controlled capacity and interface choices. Both post-hoc merging and MTL assume a fixed task suite. The setting where that suite changes over time introduces an additional challenge.

2.6 Continual Adapter Merging

In practice, the set of tasks a deployed system must handle can change over time. Classical sequential fine-tuning addresses this by continuing training on new data, but it can overwrite earlier task behaviour, a problem known as catastrophic forgetting. Approaches such as elastic weight consolidation [21] and experience replay [39] try to reduce this overwriting. Adapter storage changes the problem: if each source adapter is stored separately beside the frozen backbone, later training does not overwrite earlier adapter weights. The forgetting problem shifts from weight overwriting to composition.

The remaining challenge is that composing a growing set of adapters can still degrade earlier tasks. A new incoming task vector may require coefficients that reduce the contribution of the previous composition, or it may interact poorly with retained task vectors. This creates an acquisition–retention trade-off: the composition should incorporate the new task while preserving performance on all previously seen tasks. The weakest retained task is the key diagnostic: a composition that drives any earlier task below its pre-adaptation baseline has not achieved reliable modular reuse.

The closest prior framework is sequential model merging, where an accumulated composed adapter is updated by incorporating a new task vector without retraining earlier adapter weights [40]. This project terms that setting continual adapter merging. Chapter 8 evaluates it empirically by adding tasks one at a time and measuring both new-task acquisition and retained-task recovery.

2.7 Research Gap

Existing work covers several parts of this setting, but not the combination needed to answer the project’s central question. Speech LLMs establish a shared autoregressive decoder for diverse audio tasks, yet do not determine whether independently trained task adapters are mutually compatible. Task arithmetic provides a post-hoc merging formalism, but its standard evaluations use models with more uniform input–output structure than the seven speech tasks considered here, where acoustic evidence, response format, and evaluation metric all differ. MTL provides a pooled-data training baseline, not a mechanism for post-hoc adapter merging. Continual learning motivates changing task sets, yet does not specify how fixed task updates should be merged when new speech tasks are added.

The resulting question is whether independently trained LoRA updates for diverse speech tasks are compatible enough to merge reliably under a shared autoregressive decoder. Prior speech-adjacent merging work [41, 42] provides relevant evidence, but does not directly test this setting: a seven-task Speech LLM suite with varied response formats, supervised post-hoc coefficient calibration, and continual changes in the task set. Therefore, the experiments test behavioural modularity directly, rather than assuming that storage modularity is sufficient.

Chapter 3

Methodology

This chapter defines the notation, methods, and metrics used throughout the project: fixed LoRA task vectors around a frozen backbone, supervised coefficient calibration, cross-task interference, headroom-normalised recovery, matched MTL comparison, and continual merging.

All merge methods treat the trained source adapters as fixed updates: the backbone and source task vectors are never modified. A single-task adapter is written as $\theta_t = \theta_0 + \tau_t$, where θ_0 is the frozen backbone and τ_t is the effective LoRA task vector. A composed model is $\theta(\lambda) = \theta_0 + \sum_{t \in \mathcal{S}} \lambda_t \tau_t$, with coefficients λ_t selected on the development split. Recovery $R_i(m)$ measures the fraction of base-to-single-task headroom preserved on task i , while cross-task interference is the off-diagonal effect of $\theta_0 + \tau_j$ on task $i \neq j$, meaning the effect of applying adapter j on a task it was not trained for.

3.1 LoRA Task Adapters and Task Vectors

Let θ_0 denote the frozen backbone parameters. For each trained task t in the source task set

$$\mathcal{T} = \{\text{ASR, ER, IC, KWS, LID, SV, VS}\},$$

a single-task LoRA adapter a_t is trained on that task alone. These adapters serve two roles: they define task-specific reference performance, and they provide the task vectors used for merging. The notation here requires only that the same frozen backbone is used for all source adapters, merges, and matched MTL adapters.

For each adapted weight matrix or module k , LoRA represents adaptation as a low-rank additive update. The effective update for task t is

$$\Delta W_{t,k} = \frac{\alpha_t}{r_t} B_{t,k} A_{t,k}, \quad (3.1)$$

where $A_{t,k}$ and $B_{t,k}$ are learned low-rank factors, r_t is the rank, and α_t is the scaling factor. Throughout the project, “task vector” means the collection of effective LoRA updates

$$\tau_t = \{\Delta W_{t,k}\}_k, \quad (3.2)$$

not the stored factors separately. Equivalently, the task-adapted model is written as $\theta_t = \theta_0 + \tau_t$.

Merging is defined over effective updates $\Delta W_{t,k}$, not the stored factors: averaging $A_{t,k}$

and $B_{t,k}$ separately is not equivalent to averaging their products. The merged update at module k is therefore

$$\Delta W_{\text{merge},k} = \sum_{t \in \mathcal{S}} \lambda_t \frac{\alpha_t}{r_t} B_{t,k} A_{t,k}. \quad (3.3)$$

Implementations may compute and store explicit dense weight updates or apply weighted low-rank products during the forward pass; this affects memory and runtime only. The experiments evaluate fixed-coefficient merges of stored adapters, not compression into a single same-rank LoRA adapter.

3.2 Post-Hoc Adapter Merging

For a merge set $\mathcal{S} \subseteq \mathcal{T}$, post-hoc merging constructs a single task-vector update

$$\tau_{\text{merge}} = \sum_{t \in \mathcal{S}} \lambda_t \tau_t, \quad (3.4)$$

and evaluates $\theta_{\text{merge}} = \theta_0 + \tau_{\text{merge}}$. The backbone θ_0 and all source task vectors τ_t are fixed; the method selects only the merge coefficients.

Uniform averaging fixes $\lambda_t = 1/|\mathcal{S}|$, testing whether independently trained updates are compatible without learned weighting. Global-scalar task arithmetic keeps the summed task-vector direction fixed and selects only its magnitude:

$$\tau_{\text{merge}}(\gamma) = \gamma \sum_{t \in \mathcal{S}} \tau_t. \quad (3.5)$$

This tests whether the main failure mode is scale calibration rather than task-specific weighting.

More flexible supervised post-hoc merging methods learn task-wise or layer-wise coefficients. Task-wise merging learns one coefficient per source task. For layer-wise merging, let $\tau_{t,\ell}$ be the component of task vector t at layer or module group ℓ . The parameterisation is

$$\tau_{\text{merge}} = \sum_{\ell=1}^L \sum_{t \in \mathcal{S}} \lambda_{t,\ell} \tau_{t,\ell}. \quad (3.6)$$

Layer-wise coefficients allow the task balance to vary across layers.

Coefficient selection requires labelled development examples for the composed tasks, but does not retrain the backbone or source adapters. For the global scalar,

$$\gamma^* = \arg \max_{\gamma \in \Gamma} J_{\text{dev}} \left(\theta_0 + \gamma \sum_{t \in \mathcal{S}} \tau_t \right), \quad (3.7)$$

where Γ is the scalar search range and J_{dev} is the development-set selection objective. In the main experiments, J_{dev} is worst-task recovery with mean recovery as a tie-break, and $\Gamma = [0, 1.5]$. This is supervised coefficient calibration over fixed task vectors, not a data-free criterion; the search uses Bayesian optimisation.

For task-wise or layer-wise coefficients, supervised coefficient optimisation minimises task losses while keeping the backbone and all source task vectors fixed:

$$\lambda^* = \arg \min_{\lambda} \sum_{t \in \mathcal{T}_{\text{opt}}} \mathcal{L}_t^{\text{CE}} \left(\theta_0 + \sum_{\ell=1}^L \sum_{j \in \mathcal{S}} \lambda_{j,\ell} \tau_{j,\ell} \right). \quad (3.8)$$

Here $\mathcal{T}_{\text{opt}}$ is the set of tasks supplying optimisation batches and $\mathcal{L}_t^{\text{CE}}$ is the task-specific cross-entropy loss. The optimisation variables are the coefficients $\lambda_{j,\ell}$, not the backbone or LoRA adapter weights. Checkpoint selection uses development-set recovery, not cross-entropy, and final reporting uses held-out test scores; test data are never accessed during coefficient search or selection. Batching, balancing, and stopping rules are specified in Chapter 4.

In the main layer-wise experiments, the coefficients are parameterised as $\lambda_{t,\ell} = \beta_{\ell} w_{\ell,t}$, where $\beta_{\ell} \geq 0$ is a learned layer scale and $\sum_{t \in \mathcal{S}} w_{\ell,t} = 1$ with $w_{\ell,t} \geq 0$. Positivity is enforced by applying the softplus transform, $\text{softplus}(x) = \log(1 + \exp(x))$, to unconstrained optimisation variables. This makes task-wise weighting the special case in which the weights do not vary with ℓ , while the reported layer-wise method allows a different coefficient profile at each layer under the same fixed source task vectors. TIES and DARE are treated as secondary baselines that apply sparsity or conflict-resolution rules in task-vector space [10, 11]. They test whether coordinate-level trimming, sign resolution, or random dropping improves compatibility before learned coefficient calibration is applied.

3.3 Recovery and Interference Metrics

The trained tasks use different raw metrics: word error rate (WER) for ASR, accuracy for some classification tasks, and macro-F1 for others. These scores cannot be averaged directly without first making their direction and scale comparable. Let $s_i(m)$ be the oriented score of model m on task i , with larger values always better. Accuracy and macro-F1 are used directly; for ASR, $s_{\text{ASR}}(m) = -\text{WER}_{\text{ASR}}(m)$. Headroom-normalised recovery is

$$R_i(m) = \frac{s_i(m) - s_i(\theta_0)}{s_i(a_i) - s_i(\theta_0)}, \quad (3.9)$$

where a_i is the selected single-task reference adapter for task i . This expresses each model relative to the task-specific improvement from the frozen backbone to the reference adapter, making heterogeneous metrics comparable for summary purposes.

The denominator $H_i = s_i(a_i) - s_i(\theta_0)$ is the task-specific headroom supplied by the selected single-task adapter. Recovery is zero at the frozen backbone and one at the single-task reference; negative values are below-base, and values above one exceed the reference. It measures preserved adapter gain, not distance from a task optimum. Raw task metrics are therefore reported alongside recovery, especially where H_i is small, as for ASR.

Cross-task interference is measured by evaluating each single-task adapter on every task:

$$S_{j,i} = s_i(a_j). \quad (3.10)$$

The recovery-normalised form is

$$R_{j,i} = \frac{s_i(a_j) - s_i(\theta_0)}{s_i(a_i) - s_i(\theta_0)}. \quad (3.11)$$

The diagonal entries measure each adapter on its own task. The off-diagonal entry $R_{j,i}$ measures how the adapter trained for task j affects task i , relative to the frozen backbone and task i 's single-task reference. The matrix is asymmetric, so $R_{j,i}$ and $R_{i,j}$ are not in general equal. It measures cross-task interference throughout the project: positive entries reflect off-target improvement; negative entries capture the degradation that constitutes task overfitting. Both polarities are interpreted alongside raw task scores, label-parse failures, and invalid-output counts.

$j \rightarrow i$ denotes the source adapter trained for task j evaluated on task i , corresponding to matrix entries $S_{j,i}$ and $R_{j,i}$. The direction matters: IC $\rightarrow$ LID and LID $\rightarrow$ IC are distinct, asymmetric entries.

D_j measures how much adapter j harms other tasks on average; V_i measures how vulnerable task i is to interference from other adapters. Both summarise below-base off-diagonal recovery:

$$\begin{aligned} D_j &= \frac{1}{|\mathcal{T}| - 1} \sum_{i \neq j} \max(0, -R_{j,i}), \\ V_i &= \frac{1}{|\mathcal{T}| - 1} \sum_{j \neq i} \max(0, -R_{j,i}). \end{aligned} \quad (3.12)$$

For a merge evaluated over task set $\mathcal{E}$, the main aggregate summaries are mean recovery and worst-task recovery:

$$\bar{R}(m) = \frac{1}{|\mathcal{E}|} \sum_{i \in \mathcal{E}} R_i(m), \quad (3.13)$$

$$R_{\min}(m) = \min_{i \in \mathcal{E}} R_i(m). \quad (3.14)$$

Aggregate summaries are useful, but they do not replace per-task results: mean recovery can obscure below-base failures, while worst-task recovery can miss moderate degradation spread across many tasks.

3.4 Matched MTL Baseline

Joint MTL provides the training-based baseline for post-hoc merging. For a merge over task set $\mathcal{S}$, the matched MTL baseline is a single LoRA adapter trained on examples from the same task set, where available. It uses the same frozen backbone, LoRA capacity, prompt interface, and evaluation metrics as the corresponding merge. The seven-task MTL adapter is therefore the matched baseline for the full seven-task merge; subset comparisons require MTL adapters trained on the corresponding subsets.

Let a_ϕ denote a trainable shared LoRA adapter with parameters ϕ . The MTL objective can be written as

$$\phi^* = \arg \min_{\phi} \sum_{t \in \mathcal{S}} w_t \mathcal{L}_t(\theta_0 + a_\phi), \quad (3.15)$$

where $w_t \geq 0$ is the scalarisation weight that sets task t 's relative contribution under the chosen sampling or loss-weighting scheme. Thus MTL is a scalarised multi-objective optimisation over the shared LoRA parameter space. By contrast, the gradient-based merge objective in Equation 3.8 optimises task losses over coefficients multiplying fixed task vectors, using the task-wise and layer-wise coefficient spaces defined in Equations 3.4 and 3.6. Both are supervised, but MTL searches the full shared LoRA parameter space while calibrated merging searches a much smaller coefficient space, and they differ substantially in the volume of task data required.

MTL has access to all selected task data during training, whereas merging trains adapters independently and composes them afterwards. The MTL adapter is therefore a matched-capacity training baseline, not an upper bound. Stronger gradient-balancing or task-weighting variants are treated as limitations.

3.5 Continual Modular Composition

Continual modular composition tests whether post-hoc merging can add a new task without retraining the earlier source adapters. After k tasks, the composed update is $\tau^{(k)}$, giving $\theta^{(k)} = \theta_0 + \tau^{(k)}$. When task $k + 1$ is added, the incremental merge combines the previous composed update with the incoming task vector:

$$\tau^{(k+1)} = \lambda_{\text{prev}} \tau^{(k)} + \lambda_{k+1} \tau_{k+1}. \quad (3.16)$$

The experiments evaluate scalar and layer-wise versions of this update. The scalar variant applies the same balance between the previous composition and the incoming adapter at every layer; the layer-wise variant allows this balance to vary, giving each layer its own weighting.

The scalar variant uses global parameters $\alpha \geq 0$ and $\lambda \in [0, 1]$:

$$\tau^{(k+1)} = \alpha \left(\lambda \tau^{(k)} + (1 - \lambda) \tau_{k+1} \right). \quad (3.17)$$

The layer-wise variant replaces these with per-layer parameters $\alpha_\ell \geq 0$ and $\lambda_\ell \in [0, 1]$:

$$\tau_\ell^{(k+1)} = \alpha_\ell \left(\lambda_\ell \tau_\ell^{(k)} + (1 - \lambda_\ell) \tau_{k+1, \ell} \right). \quad (3.18)$$

Both variants select coefficients using supervised feedback over the seen tasks, while keeping the source task vectors fixed. The layer-wise variant reduces to the scalar case when the same α and λ are used across all layers.

Compressing previous tasks into $\tau^{(k)}$ means that later stages cannot independently rebalance all individual earlier adapters without re-optimising over their task vectors. Continual success is therefore judged by both new-task acquisition and weakest retained-task recovery: a method that learns the incoming task while driving a previous task below the frozen-backbone baseline has not achieved satisfactory modular composition.

Chapter 4

Experimental Design

Chapter 3 defined the notation and metrics. The backbone, training capacity, prompts, and evaluation rules are held constant throughout, so that comparisons between methods reflect training or merging decisions rather than evaluation choices.

4.1 Backbone, Tasks, and Evaluation Metrics

All experiments use Qwen2.5-Omni-3B as the common speech-language backbone [3]. The backbone is frozen throughout. Single-task and MTL experiments train LoRA adapters on top of it, while post-hoc merges combine fixed adapter updates. Holding the backbone fixed controls for pretraining and architecture variation across experiments.

Table 4.1 defines the trained task suite, data splits, output format, and primary metric used throughout the experiments.

Task	Dataset	Output / evidence	Train	Val.	Test	Metric
ASR	LibriSpeech [14]	Transcript; lexical	28,539	2,703	2,642	WER ↓
ER	MELD [17]	Label; prosody	9,678	1,070	2,524	F1 ↑
IC	SLURP [15]	Label; semantics	50,626	8,689	13,076	Acc ↑
KWS	Speech Commands [16]	Label; keywords	84,653	9,956	4,482	F1 ↑
LID	FLEURS [20]	Label; language	25,973	3,265	6,878	Acc ↑
SV	VoxCeleb2 [19]	Binary; speaker	16,000	2,000	2,000	Acc ↑
VS	VocalSound [18]	Label; vocal sounds	14,759	1,763	3,409	Acc ↑

Table 4.1: Seven trained tasks, split sizes, and primary metrics, fixed for all training, merging, MTL, and evaluation runs throughout the experiments. Output/evidence characterises the output format and primary observable signal.

The suite spans free-form transcription, short-label classification, binary verification, and vocal sound classification, and tests whether fixed-coefficient merging preserves performance across tasks with substantially different output forms.

WER is negated when computing recovery summaries (lower is better). Macro-F1 is used for ER and KWS because class balance matters; accuracy is used for IC, LID, SV, and VS. Final results report raw per-task metrics alongside mean and worst-task recovery, since aggregate recovery alone can hide below-base degradation. Raw ASR WER is always reported alongside recovery because the ASR base-to-single-task headroom is narrow. The ASR validation and test splits are LibriSpeech `dev-clean` and `test-clean`; all main-

body ASR results use `test-clean`. The supplementary `test-other` check is reported in Section 7.3.

Speaker verification uses constructed VoxCeleb2 pairs, with speakers split before pair construction so that validation and test identities are disjoint from training. Pairs are class-balanced, and the two clips are concatenated with a short silence gap before the binary prompt. SV is evaluated as a prompted yes/no verification task, reporting accuracy for consistency with the other classification tasks. It is not a standard automatic speaker verification benchmark: scores are not thresholded and EER is not reported. The task probes held-out speaker identity using a paired-audio input format that Qwen2.5-Omni was not specifically pretrained for.

Spoken question answering (SQA), evaluated using MMSU [43], is used only as a held-out diagnostic. It is not a trained source task, supplies no coefficient-selection or matched MTL data, and is excluded from mean and worst-task recovery. Chapter 7 reports the overall diagnostic; Appendix A.2 gives the category-level detail.

4.2 Prompting, Decoding, and Output Parsing

All tasks are formulated as audio-conditioned causal language modelling: the model receives audio and a task instruction, then generates a text response without any classification head. As a result, off-target degradation can appear either as a parseable label error or as invalid-output drift.

Prompts are fixed by task and list valid labels where applicable. ASR uses beam search for free-form transcription; classification, binary, and multiple-choice tasks use deterministic short-form decoding. The same prompt, decoding rule, parser, and normalisation procedure are used for the frozen backbone, single-task adapters, fixed-coefficient compositions, and matched MTL adapters, so all compared methods run under identical evaluation conditions, with prompt, decoding, parsing, and normalisation settings strictly controlled.

Generated outputs are parsed strictly so that task failures are not hidden by permissive post-processing. Classification and binary tasks are matched against the allowed label inventory; SV expects `yes/no`; SQA expects one option letter from `{A, B, C, D}`. Outputs that cannot be matched are counted as invalid and incorrect. ASR outputs are text-normalised before WER computation.

4.3 Training and Capacity Controls

Each source task has one independently trained LoRA adapter. These adapters serve as the single-task references for computing recovery and provide the task vectors used by post-hoc merging. Single-task runs use the frozen backbone, validation-based early stopping, and restoration of the best validation checkpoint before final test evaluation. Learning rates and epoch budgets were chosen by validation performance; the single-task references are therefore validated task-specific baselines rather than fully optimised upper bounds.

All trained adapters use the same LoRA configuration [6]: rank $r = 64$, scaling $\alpha = 128$, dropout 0.1, no LoRA bias terms, and LoRA applied to the attention, MLP, and audio-projector modules. Fixing this configuration ensures all adapters have the same parameter budget across single-task, merge, and MTL configurations. Under this configuration, the reported source and matched MTL adapters have 164.6M trainable LoRA parameters and require approximately 314 MiB per checkpoint. Retaining all adapters allows different subsets and coefficient profiles to be evaluated without retraining. Post-merge compression is not evaluated.

Task	LR	Max epochs	Selection metric
ASR	2×10^{-5}	3	WER
ER	2×10^{-5}	5	Macro-F1
IC	3×10^{-5}	3	Accuracy
KWS	5×10^{-5}	5	Macro-F1
LID	3×10^{-5}	1	Accuracy
SV	3×10^{-5}	3	Accuracy
VS	3×10^{-5}	3	Accuracy

Table 4.2: Single-task LoRA source-adapter training settings. Epochs are maximum budgets; checkpoint selection uses the listed validation metric.

4.4 Merging and Matched MTL Protocols

Three post-hoc merging approaches are evaluated. Uniform delta averaging sets $\lambda_t = 1/|\mathcal{S}|$ for each task, applying no learned weighting. Global-scalar task arithmetic selects a single coefficient γ over the summed task vectors $\gamma \sum_t \tau_t$, chosen on the development split to maximise worst-task recovery. Supervised coefficient calibration estimates task-wise or layer-wise coefficients over fixed task vectors. Task-wise coefficients sit between a single global coefficient and full per-layer tuning; the main supervised merge reported in Chapter 6 is Layer-wise, because it allows task weights to vary across layers while retaining fixed source task vectors. TIES and DARE are included as sparsity-oriented secondary diagnostic baselines [10, 11], not as the central full-suite methods.

For scalar task arithmetic, γ is selected on a development split distinct from the validation split used for single-task checkpointing, by maximising worst-task recovery, with mean recovery as a secondary tie-break. For supervised coefficient methods, coefficients are estimated on the development split while the single-task vectors remain fixed, and checkpoints are selected by development-set recovery. This development split is drawn from examples held out of each task’s training data, with up to 500–1000 unique labelled examples per task; it is used only for coefficient selection, while source-adapter checkpoint selection uses the validation split and final reporting uses the test split.

Gradient-based merging applies weighted LoRA products directly during the forward pass rather than computing explicit dense weight updates, with LoRA factors retained so that the merged update remains a weighted sum of low-rank products. The reported

memory and wall-clock measurements were taken on a single NVIDIA RTX 6000 Ada GPU with 48 GiB of VRAM. In the seven-task merge, measured CUDA allocator peak memory usage was 2.146 GiB for the fused LoRA forward path and 44.495 GiB for the dense-matrix approach, a 20.7-fold peak-memory reduction; this changes memory use, not the mathematical merge.

The methods differ greatly in the number of merge-side variables. Uniform has no trainable merge variables; Scalar selects one global coefficient; and Layer-wise optimises $T+LT+1+L$ trainable scalar variables controlling task weights and layer magnitudes. For the seven-task, 36-layer merge this gives 296 scalar variables over fixed task vectors, compared with 164.6M trainable LoRA parameters in the matched MTL run. The optimisation volume also differs: the full-suite Layer-wise run processes 14,400 examples, counting gradient-step cycling over the unique per-task subsets, while the matched MTL run processes 230,228 training examples. The corresponding wall-clock times were 5.65 hours and 19.4 hours respectively, but these timings include validation evaluation and checkpoint-selection overhead, and should not be read as a normalised compute comparison.

MTL baselines are matched to the merge set being evaluated. The seven-task MTL adapter is used for the full-suite comparison, while subset merges are compared with MTL adapters trained on the same task set for the subset settings reported in Chapter 7. Subset experiments without a matched MTL artefact are reported as merge-only analyses rather than direct MTL comparisons. All MTL baselines use the same backbone, LoRA capacity, target modules, prompts, decoding rules, parsers, and metrics as the single-task adapters and fixed-coefficient compositions. Batches are homogeneous by task, and temperature-based task sampling reduces domination by the largest datasets. Both configurations are matched in capacity, though neither is fully optimised for its task set.

Method	Coefficient selection	Role
Uniform	Fixed $\lambda_t = 1/ \mathcal{S} $	Uncalibrated baseline
Scalar	Bayesian optimisation over $\gamma \sum_t \tau_t$, $\gamma \in [0, 1.5]$	Global-magnitude calibration
Layer-wise	Supervised cross-entropy optimisation of $\beta_\ell w_{\ell,t}$; source task vectors fixed	Main coefficient-optimised merge
MTL	Pooled-data training of a shared LoRA adapter on the same task set	Matched training baseline
TIES / DARE	Sparsity and sign/conflict rules	Secondary conflict-aware baselines

Table 4.3: Post-hoc merge methods and matched MTL baseline. The Role column states the function of each configuration in the experimental design.

4.5 Continual Protocol

Both continual coefficient-space methods use a custom gradient-based coefficient optimiser. Source task vectors remain fixed while trainable coefficients are optimised over seen tasks

using supervised cross-entropy with equal task weighting. Checkpoints are selected by mean recovery on the development split.

Coefficients are constrained positive via the softplus transform $\log(1 + \exp(x))$ for α and to $[0, 1]$ via sigmoid for λ . Two-scalar optimises global α and λ ; Layer-wise continual extends this with per-layer pairs $(\alpha_\ell, \lambda_\ell)$. Layer-wise continual is the main method in Chapter 8; Two-scalar is the scalar gradient baseline. Both are custom implementations rather than published merging methods.

The five diagnostic task orders (Table 8.1) probe four distinct conditions: acoustic growth, output-format shifts, ASR transfer without ASR training, and order sensitivity. The last is examined through the semantic-first versus speaker-first reversal pair.

For continual merging, the previous merge becomes the prior after the second stage, and each later stage optimises coefficients that combine the next single-task adapter with that prior over the seen tasks. Sequential MTL is the training-based baseline: it continues optimisation from the previous shared-adapter checkpoint as tasks are added, updating shared LoRA parameters over the seen task set. Therefore, the comparison separates recombining fixed task vectors from continued shared-adapter training.

4.6 Reproducibility Scope

The reported tables use the primary development-set-selected configuration for one Qwen2.5-Omni-3B backbone. Across seeds, prompt variants, LoRA ranks, target-module choices, and optimisation settings,¹ the qualitative ordering was consistent: naive averaging remained the weakest approach, Scalar calibration produced the largest improvement, and Layer-wise calibration achieved the strongest task balance.

Merge coefficients are selected on the development split, separate from the held-out test split. Calibrated merging is therefore post-hoc over fixed source adapters but supervised in coefficient selection: the optimised parameters are merge coefficients over frozen task vectors, not model weights.

With this setup established, Chapter 5 first examines whether the independently trained source adapters act as independent task modules, each affecting only its own task, or instead exhibit structured off-task interference.

¹The checks were exploratory rather than a balanced sensitivity study, so they inform qualitative confidence but are not reported as statistical robustness evidence.

Chapter 5

Cross-Task Adapter Interference

Each source adapter is individually competent: it improves its own task relative to the frozen backbone. The question is whether that competence is isolated. If each adapter affected only its own task, evaluating it outside its training scope would leave other tasks unaffected. To test this, each adapter is evaluated on all seven tasks.

The adapters are not isolated. Table 5.2 shows directional, task-pair-specific off-task effects. The harmful cases are task overfitting, as defined in Chapter 1, while beneficial off-task effects coexist with harmful ones in the same interference matrix. Later sections examine whether the largest classification failures are parsing artefacts and whether simple weight-space diagnostics explain the compatibility pattern. Chapter 6 treats merging as an empirical calibration problem.

Notation follows Chapter 3: τ_t is the task vector; $R_{j,i}$ measures cross-task recovery for a specific adapter–task pair; D_j and V_i summarise outgoing disruption and incoming vulnerability across all pairs. Parseable label errors assign valid labels incorrectly; invalid-output errors are unrecognised or unparseable.

5.1 Single-Task and Cross-Task Adapter Behaviour

Task	Metric	Base	Single-task	Improvement
ASR	WER ↓	2.35%	2.02%	0.33 pp
ER	Macro-F1 ↑	31.9%	45.7%	13.8 pp
IC	Accuracy ↑	54.0%	85.5%	31.5 pp
KWS	Macro-F1 ↑	88.8%	92.2%	3.4 pp
LID	Accuracy ↑	88.4%	99.9%	11.5 pp
SV	Accuracy ↑	50.9%	92.6%	41.7 pp
VS	Accuracy ↑	85.0%	94.5%	9.5 pp

Table 5.1: Base model and single-task adapter performance on each of the seven trained tasks. Every adapter improves its own task; pp denotes the absolute percentage-point change from Base to Single-task.

Table 5.1 establishes the starting condition: every source adapter improves its own task. The cross-task failures shown in Table 5.2 are compatibility failures, not failed fine-tuning runs. Improvement size varies substantially: ASR and KWS mainly refine an already capable base, whereas IC and SV substantially improve weaker base performance. Evaluating each task vector τ_t on $t' \neq t$ then tests whether each adapter affects only its

own task. If the adapters were clean modules, off-diagonal cells would remain close to the frozen backbone. Table 5.2 shows that they do not.

Source adapter	ASR (WER ↓)	ER (F1 ↑)	IC (Acc ↑)	KWS (F1 ↑)	LID (Acc ↑)	SV (Acc ↑)	VS (Acc ↑)
Base	2.35	31.9	54.0	88.8	88.4	50.9	85.0
ASR	2.02	29.2	44.6	88.7	93.8	53.4	86.1
ER	2.15	45.7	56.0	88.9	92.8	51.9	83.9
IC	3.22	34.1	85.5	89.3	59.5	58.1	84.2
KWS	2.22	29.8	47.7	92.2	75.5	52.4	87.2
LID	2.16	29.6	55.5	89.0	99.9	51.1	85.2
SV	2.04	30.0	50.7	89.7	97.8	92.6	88.1
VS	1.92	30.0	42.7	89.5	99.1	65.8	94.5

Table 5.2: Cross-task raw metrics for all seven source adapters. Diagonal cells (blue) show on-task performance; green and red off-diagonal cells mark gains and losses relative to the frozen base. The most severe failures are IC→LID (59.5%) and KWS→LID (75.5%), both parseable-label degradations; most other off-diagonal perturbations are small.

Three properties matter for merging. First, damage is concentrated rather than diffuse. The clearest failures are IC→LID and KWS→LID, both severe parseable-label degradations: IC reduces LID accuracy from 88.4% to 59.5%, and KWS reduces it to 75.5%. IC also raises ASR WER from 2.35% to 3.22%. Most other off-diagonal cells are smaller perturbations, so cross-task interference is task-pair-specific.

Second, interference is mixed with beneficial off-task effects, which is why calibrated merging is worth attempting. Five of the seven adapters preserve or improve at least four of their six off-task evaluations. VS reduces ASR WER to 1.92%, below the ASR adapter’s own 2.02%, and raises LID accuracy to 99.1%. SV also preserves ASR and improves LID. Therefore, the source adapter set contains reusable off-target effects alongside harmful interference.

Third, no adapter is uniformly benign or harmful. VS improves ASR and LID but drops IC accuracy to 42.7%. IC excels on its own task but is the most damaging source for ASR and LID. The relevant unit for composition is the adapter–task pair, not the adapter in isolation.

5.2 Asymmetry and Parsing Diagnostics

Interference is asymmetric: Figure 5.1 separates outgoing disruption D_j from incoming vulnerability V_i to make this directionality explicit.

IC is the clearest high-disruption source; LID the clearest vulnerable target. SV occupies the stable region (every non-source adapter leaves it at or above the backbone). VS shows why the two-axis view is useful: it is broadly reusable for ASR and LID, yet is the main failure source for IC.

IC degrades ASR and LID, but the reverse is not true: LID leaves ASR WER at 2.16%, and ASR raises LID accuracy to 93.8%. ER has several below-base incoming cells, but the

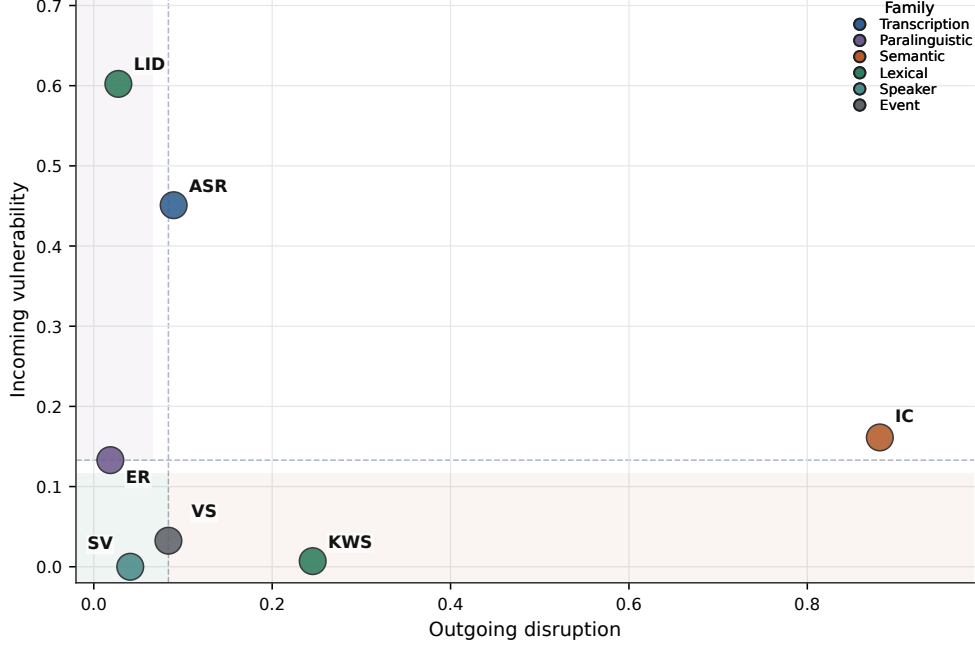


Figure 5.1: Task compatibility map derived from off-diagonal recovery. The x-axis is outgoing disruption D_j by source adapter; the y-axis is incoming vulnerability V_i of the target task. IC occupies the high-disruption region; LID the high-vulnerability region; SV the stable corner.

absolute losses are modest relative to the LID failures.

Because all non-ASR tasks are evaluated by parsing generated text, large classification drops could in principle be output-format failures rather than incorrect task decisions. Table 5.3 focuses on the highest-impact off-task degradations from Table 5.2 and separates parseable label errors from invalid-output errors. These cases are not an exhaustive error taxonomy but make the aggregate interference pattern concrete.

Eval task	Source adapter	Metric	Recognised	Interpretation
LID	IC	59.5% acc.	99.9%	Parseable label errors.
LID	KWS	75.5% acc.	100.0%	Parseable label errors.
IC	VS	42.7% acc.	84.2%	Partly invalid outputs.
ER	ASR	29.2% F1	100.0%	No parse failure.

Table 5.3: Output-side diagnostics for selected off-task failures. Recognised is the rate of parseable task labels. Rows represent selected high-impact off-task degradations from Table 5.2. The two LID failures are valid-label errors rather than parser artefacts; VS→IC is the exception, with a 15.8% unrecognised-output rate indicating a distinct format-sensitivity failure.

The two LID failures are valid-label errors. IC→LID and KWS→LID have recognised-label rates of 99.9% and 100.0%, so the model is producing parseable LID labels but assigning them incorrectly. The confusion matrices in Appendix A.3 make this visible: both failures over-predict English for several non-English utterances, most strongly for Japanese under KWS (59.5%) and for Japanese, Russian, and Chinese under IC (54.2%, 53.4%, and 50.8%). This points to a shifted label prior rather than parser failure.

The VS→IC failure is different. Its 15.8% unrecognised-output rate means that part of

the accuracy drop comes from invalid outputs: VS adaptation shifts output templates towards sound-category vocabulary that partially overlaps with, but does not fully match, the intent label set. ER under the ASR adapter is parseable but weak. Therefore, the LID failures are genuine label-prior shifts rather than parsing artefacts; VS→IC represents a distinct format-sensitivity mode.

5.3 Weight-Space Diagnostics

Global task-vector geometry does not explain the behavioural matrix. Following Equation 3.1, the per-module effective update is $\Delta W_{t,k} = (\alpha_t/r_t)B_{t,k}A_{t,k}$.

For diagnostics, each task vector is flattened into $\mathbf{v}_t = \text{concat}_k \text{vec}(\Delta W_{t,k})$. The L2 norm is $n_t = \|\mathbf{v}_t\|_2$, and cosine similarity with task u is $\rho_{t,u} = \mathbf{v}_t^\top \mathbf{v}_u / (\|\mathbf{v}_t\|_2 \|\mathbf{v}_u\|_2)$.

Pairwise $\rho_{t,u}$ values between the seven task vectors range from -0.003 to 0.045 , near zero across all off-diagonal pairs. This negative result is informative: global vector alignment does not account for the observed cross-task interference. IC causes the largest LID and ASR failures despite near-zero cosine similarity with those task vectors, so compatibility is not captured by global alignment alone.

Layer-local checks do not change the conclusion: the major failure pairs do not stand out geometrically even within individual decoder blocks. Layer-local cosine uses the same definition restricted to modules in a layer or module group. Early decoder layers show the largest off-diagonal spread, with mean absolute cosine similarity 0.035 and maximum 0.103 , above the global mean absolute cosine similarity of 0.012 . The audio projection is also somewhat less orthogonal, with mean absolute cosine similarity 0.027 . IC–LID has early-layer cosine similarity 0.017 , IC–ASR 0.011 , and KWS–LID 0.028 , none of which stands out among off-diagonal pairs. Correlations between block-wise cosine similarity and both outgoing disruption (D_j) and incoming vulnerability (V_i) remain weak, so compatibility cannot be inferred from simple global or layer-local geometry.

The norms n_t span 1.553 for LID to 9.334 for IC, a six-fold spread, with ASR (5.525) and SV (5.571) also substantially larger than LID. This spread makes global scale a plausible failure mode for naive composition, but norm alone does not predict compatibility.

5.4 Implications for Merging

The source adapter set is competent task-by-task, but cross-task evaluation reveals directional, task-pair-specific effects. Beneficial and harmful off-task effects coexist in the same adapter set, making calibrated merging worth testing rather than ruling out composition entirely. The weight-space diagnostics do not explain the interference pattern, but the six-fold norm spread makes global scale the first failure mode to address. Near-zero pairwise cosine similarities (global mean absolute 0.012) show that task-vector directions do not systematically cancel, so the immediate problem is poorly calibrated combined magnitude rather than wrong direction. This motivates Scalar calibration in Chapter 6 before Layer-wise tuning.

Chapter 6

Static Model Merging

Chapter 5 established that independently trained speech LoRA adapters produce structured, directional cross-task interference rather than acting as independent task modules. Weight-space diagnostics revealed a six-fold norm spread alongside near-zero pairwise cosine similarities, making global scale the first failure mode to address. The question here is which post-hoc calibration mechanisms can convert these fixed source task vectors into a usable multi-task system.

The results form a clear hierarchy. Uniform is insufficient on this seven-task suite; TIES and DARE do not close the gap in the evaluated configurations. A single scalar coefficient recovers much of the performance lost by naive merging, showing that task-vector directions remain useful but their raw summed magnitude is poorly calibrated. Layer-wise achieves the highest mean and worst-task recovery and is the strongest static merge method evaluated on this suite.

ER remains the lowest-recovered task across calibrated methods and is the main remaining bottleneck. For ASR, the headroom-normalised score exceeds one across all merges and must be read against raw WER rather than taken at face value. Leave-one-out tests show that merging composes available adapter information but cannot supply coverage for a missing source.

Five methods are compared across the full seven-task suite: Uniform, TIES, and DARE as conflict-aware baselines, and Scalar and Layer-wise as supervised post-hoc calibration methods (Table 4.3). The comparison with joint multi-task learning is deferred to Chapter 7.

6.1 Calibration Method Selection

Scalar is the simplest calibrated method that meaningfully improves on Uniform averaging. It applies a single global coefficient γ to the task-vector set: $\gamma \sum_t \tau_t$. Since it leaves relative task balance unchanged, it tests whether the merge failure is primarily a mis-scaling problem. Uniform corresponds to $\gamma = 1/|\mathcal{S}|$ in this parameterisation; optimising γ asks whether a data-selected global scale corrects this failure mode. Bayesian optimisation (BO) selects $\gamma = 0.585$ over $[0, 1.5]$ by maximising development-set worst-task recovery, with mean recovery as a tie-break. The selected value is well above the Uniform-equivalent $1/7 \approx 0.143$ and below the raw sum ($\gamma = 1$), consistent with Chapter 5’s finding that task-vector directions are useful but substantially norm-imbalanced.

Layer-wise optimises per-layer coefficients $\lambda_{t,\ell} = \beta_\ell w_{\ell,t}$, as defined in Section 3.2. Only

merge coefficients are trained; the source task vectors remain fixed. Model selection uses worst-task recovery on the development split, and final results use the held-out test split.

Figure 6.1 shows the development-split searches. The Scalar sweep peaks between Uniform and the raw sum, confirming that neither default is optimal and that global magnitude is a meaningful calibration variable. The Layer-wise search reaches its selected checkpoint at step 800, after which the selection metric does not improve.

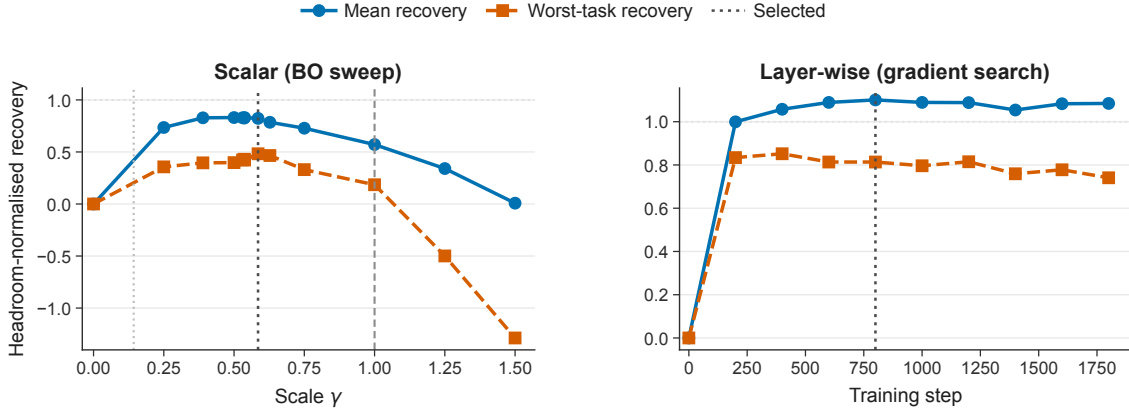


Figure 6.1: Development-split calibration searches for the seven-task suite. The left panel varies the Scalar coefficient γ ; the right panel tracks Layer-wise coefficient optimisation by training step. Dotted vertical lines mark the selected $\gamma = 0.585$ and checkpoint 800, chosen by calibration worst-task recovery with mean recovery as tie-break where applicable.

6.2 Seven-Task Simultaneous Merge

The seven-task merge combines the ASR, ER, IC, KWS, LID, SV, and VS task vectors into one fixed-coefficient merge. Both absolute metrics and recovery are reported: raw values give the achieved performance level; recovery measures the fraction of single-task improvement preserved in the merged model.

Model	ASR (WER $\downarrow$)	ER (F1 $\uparrow$)	IC (Acc $\uparrow$)	KWS (F1 $\uparrow$)	LID (Acc $\uparrow$)	SV (Acc $\uparrow$)	VS (Acc $\uparrow$)
Base	2.35	31.9	54.0	88.8	88.4	50.9	85.0
Single-task	2.02	45.7	85.5	92.2	99.9	92.6	94.5
Uniform	1.90	34.1	64.7	89.8	97.6	65.5	90.2
TIES	1.81	37.7	81.4	91.8	96.3	83.7	93.1
DARE	1.88	34.2	66.6	89.8	98.3	67.0	90.7
Scalar	1.72	37.1	80.1	92.2	98.9	86.0	92.6
Layer-wise	1.81	42.6	83.9	92.1	98.6	89.8	93.8

Table 6.1: Seven-task absolute metrics in percent. Base and Single-task rows are references; bold marks the best post-hoc merge per column.

Tables 6.1 and 6.2 give the main seven-task result. Uniform and DARE show that naive merging is weak: they produce nearly identical profiles with ER below 35% macro-F1 and

IC and SV both below 67%. DARE’s stochastic sparsification does not materially change the default result.

TIES is stronger than Uniform and DARE on some tasks, recovering IC (81.4%) and SV (83.7%), but it does not solve the merging problem. Its LID score falls to 96.3%, below Uniform’s 97.6%, and its $R_{\min} = 0.424$ remains far below Layer-wise’s worst-task recovery. Therefore, coordinate-level conflict filtering is insufficient here: it does not fix the scale and task-balance errors that calibrated methods address.

A follow-up scalar check sharpens this interpretation. TIES selected scale 1.05, giving slightly higher mean recovery but lower held-out worst-task recovery than the reported default TIES row. DARE was more scale-sensitive: at effective scale 0.600, held-out recovery rose from $R_{\min} = 0.167$, $\bar{R} = 0.595$ to $R_{\min} = 0.374$, $\bar{R} = 0.897$. This supports the global-scale failure diagnosis, but calibrated DARE still remains below Scalar in mean recovery and below Layer-wise on both aggregate criteria.

Scalar gives the main calibration jump over the uncalibrated methods. It raises IC to 80.1%, KWS to 92.2%, SV to 86.0%, and $\bar{R}_{\text{ASR}}$ to 0.793, while using only one coefficient. This confirms that much of the merge failure is a scale problem, not complete task-vector incompatibility.

Layer-wise raises ER to 42.6% macro-F1, SV to 89.8%, $R_{\min}$ to 0.773, and $\bar{R}$ to 1.011. The advantage is not uniform across every raw task metric: Scalar gives the lowest ASR WER and is slightly stronger on KWS and LID, while Layer-wise is strongest on ER, IC, SV, VS, and the mean and worst-task recovery. Layer-wise is thus the strongest static merge method evaluated on this suite by both mean and worst-task recovery. Scalar is the simpler option, recovering substantial performance with a single coefficient and achieving the lowest raw ASR WER.

Method	ASR	ER	IC	KWS	LID	SV	VS	$R_{\min}$	$\bar{R}$
Uniform	1.373	0.156	0.338	0.285	0.801	0.351	0.551	0.156	0.551
TIES	1.680	0.424	0.869	0.857	0.685	0.787	0.861	0.424	0.880
DARE	1.451	0.167	0.397	0.301	0.856	0.387	0.607	0.167	0.595
Scalar	1.936	0.375	0.829	0.994	0.911	0.842	0.805	0.375	0.956
Layer-wise	1.652	0.773	0.947	0.962	0.883	0.934	0.929	0.773	1.011

Table 6.2: Headroom-normalised recovery for the main seven-task merge. Recovery of 1.0 indicates full single-task performance recovered; $R_{\min}$ is the weakest per-task recovery and $\bar{R}$ is the mean over the seven trained tasks.

ER remains the lowest-recovered task across all calibrated methods. Its recovery rises from 0.156 (Uniform) and 0.167 (DARE), through 0.375 (Scalar) and 0.424 (TIES), to 0.773 (Layer-wise), yet it is not fully recovered even under the best method. Calibration repairs much of the cross-task interference observed in Chapter 5, but ER remains the bottleneck. A plausible explanation is that MELD emotion recognition is a noisier, class-imbalanced macro-F1 task, so its task vector may encode a narrower decision boundary than cleaner label tasks such as LID or KWS.

6.2.1 Layer-wise Coefficient Structure

Table 6.3 groups the effective coefficients $\lambda_{t,\ell} = \beta_{\ell} w_{\ell,t}$ into early, middle, and late layers and records which task holds the largest coefficient at each layer. These coefficients describe where task-vector compatibility varies across the network; they are not direct mechanistic evidence of where task knowledge is stored.

Task	Early	Mid	Late	Dom. layers	Dom. share	Recovery
ASR	0.405	0.665	0.625	3	8%	1.652
ER	0.704	0.544	0.811	11	31%	0.773
IC	0.697	0.832	0.813	16	44%	0.947
KWS	0.396	0.383	0.481	0	0%	0.962
LID	0.485	0.387	0.573	1	3%	0.883
SV	0.760	0.661	0.504	5	14%	0.934
VS	0.415	0.402	0.457	0	0%	0.929

Table 6.3: Layer-wise coefficient analysis, seven-task merge. The grouping is deliberately coarse; individual layer maxima should not be over-interpreted. *Early/Mid/Late*: mean effective coefficient $\beta_{\ell} w_{\ell,t}$ in layers 1–12, 13–24, and 25–36. *Dom. layers*: number of the 36 layers at which that task holds the largest coefficient; *Dom. share* is the corresponding fraction. *Recovery* is the headroom-normalised test recovery of the final seven-task Layer-wise merge.

The selected coefficient structure is not a simple early-acoustic/late-semantic split. SV has the largest mean early-layer coefficient (0.760), but ASR does not dominate the early blocks and is largest in only three layers. IC has the largest selected coefficients overall, especially in middle and late layers, and dominates 16 of 36 layers. KWS and VS never hold the largest coefficient at any layer, yet both recover well (0.962 and 0.929), showing that coefficient dominance is not required for good recovery. These coefficients describe how the selected merge uses the task vectors; they should not be read as a causal map of where each task is encoded in the network.

Despite receiving the second-highest mean selected coefficient (0.686) and dominating 11 layers, ER achieves only 0.773 recovery. High coefficient assignment is therefore not sufficient to close the gap, consistent with ER being more sensitive to composed interference than the cleaner classification tasks.

6.2.2 ASR Raw WER and Recovery

ASR recovery exceeds one across all merged models because the base-to-adaptor headroom is narrow (0.33 percentage points), so it should be read alongside raw WER. All five merged models improve on the single-task ASR adaptor’s 2.02% WER: the range is 1.72% for Scalar to 1.90% for Uniform, with TIES and Layer-wise at 1.81% and DARE at 1.88%.

Table 6.4 provides the diagnostic breakdown. The ASR-only scalar row shows that the ASR adaptor benefits from scale reduction: applying $\gamma = 0.417$ to the ASR task vector alone already improves WER from 2.02% to 1.81%. The full seven-task Scalar merge has effective ASR projection ($c_{\text{ASR}} = 0.611$, in ASR-task-vector norm units) and cosine alignment with the ASR task vector ($\rho_{\text{ASR}} = 0.391$), while Layer-wise has $c_{\text{ASR}} = 0.609$ and

Diagnostic	c_{ASR}	ρ_{ASR}	WER $\downarrow$
Single-task ASR adapter	1.000	1.000	2.02
ASR-only scalar ($\gamma = 0.417$)	0.417	1.000	1.81
Seven-task Scalar	0.611	0.391	1.72
Seven-task Layer-wise	0.609	0.355	1.81

Table 6.4: ASR mechanism diagnostics. c_{ASR} reports ASR-direction projection in ASR-task-vector norm units, and ρ_{ASR} reports cosine alignment with the ASR task vector. Bold marks the lowest WER.

$\rho_{\text{ASR}} = 0.355$. The merged update combines task vectors pointing in different directions, and part of the ASR WER improvement is consistent with correcting the ASR adapter’s over-large contribution.

Appendix A.4 gives the pairwise ASR check supporting the same WER analysis. The ASR-only scalar comparison improves WER from 2.18% to 1.83% on the same-sample dev-clean comparison set: 296 utterances improve, 186 worsen, and 2221 are unchanged. Several ASR-containing merges in the same appendix show the same broad WER-improving direction; these checks give useful supporting context, not a robustness claim.

The conclusion is narrow but important: ASR improves in raw error terms, and for ASR the recovery score mainly measures preservation of a relatively small adapter improvement.

6.3 Leave-One-Out Adapter Coverage

Leave-one-out (LOO) analysis shows that calibrated merging draws on available adapter information but cannot substitute for a missing source. Each source adapter is omitted in turn; the remaining six are re-merged using coefficients selected on the corresponding development split, and the omitted task is evaluated using only those vectors. Coverage varies widely across tasks, from near-full for ASR to strongly negative for ER and IC.

Omitted	ASR	ER	IC	KWS	LID	SV	VS	$R_{\min}$	$\bar{R}$
Layer-wise (7T)	1.652	0.773	0.947	0.962	0.883	0.934	0.929	0.773	1.011
<i>One source adapter omitted</i>									
ASR	<i>0.657</i>	0.635	0.956	0.911	0.913	0.959	0.932	0.635	0.884
ER	1.686	<i>-0.054</i>	0.891	1.027	0.938	0.871	0.947	0.871	1.060
IC	1.652	0.214	<i>-0.516</i>	1.034	0.987	0.910	0.916	0.214	0.952
KWS	1.618	0.717	0.913	<i>0.565</i>	0.957	0.915	0.938	0.717	1.010
LID	1.692	0.490	0.892	1.088	<i>0.291</i>	0.872	0.947	0.490	0.997
SV	1.504	0.289	0.887	1.055	0.887	<i>0.441</i>	0.898	0.289	0.920
VS	1.350	0.661	0.926	0.947	0.938	0.890	<i>0.025</i>	0.661	0.952

Table 6.5: Layer-wise leave-one-out recovery. Each row omits one source adapter; italicised entries are the omitted task’s recovery from the remaining six adapters, and $R_{\min}$ and $\bar{R}$ exclude the omitted task.

Table 6.5 shows the Layer-wise coverage test. Across in-merge tasks, ER remains the usual worst task; omitting ER lifts $R_{\min}$ to 0.871, now limited by SV at 0.871. Across the other omission settings, $R_{\min}$ ranges from 0.214 when IC is omitted to 0.717 when KWS is omitted.

The omitted-task column reveals four coverage patterns. ASR is partially supported by the remaining adapters, reaching $R = 0.657$ (WER $\approx 2.13\%$) without its own adapter; because ASR headroom is narrow, this recovery should be read together with the raw WER. KWS, SV, and LID have partial coverage, at 0.565, 0.441, and 0.291 respectively. VS is essentially uncovered at $R = 0.025$. ER and IC are more severe: when omitted, they fall below the frozen backbone at $R = -0.054$ and $R = -0.516$. In both cases, the remaining task vectors displace, rather than recover, the relevant behaviour.

Layer-wise gives better $R_{\min}$ in four of seven omission settings (Table 6.6), with the clearest margins for ASR-omitted (0.635 versus 0.461) and KWS-omitted (0.717 versus 0.538). Scalar is stronger for SV-omitted (0.402 versus 0.289); the IC-omitted and VS-omitted settings are near-tied. The Layer-wise advantage thus extends beyond the full seven-task merge, but it depends on the adapter set containing the necessary source information.

Omitted	Scalar $R_{\min}$	Scalar $\bar{R}$	Layer-wise $R_{\min}$	Layer-wise $\bar{R}$
ASR	0.461	0.845	0.635	0.884
ER	0.840	1.071	0.871	1.060
IC	0.220	0.983	0.214	0.952
KWS	0.538	1.002	0.717	1.010
LID	0.465	0.966	0.490	0.997
SV	0.402	1.007	0.289	0.920
VS	0.670	1.029	0.661	0.952

Table 6.6: Scalar versus supervised layer-wise leave-one-out recovery. Each row omits one source adapter; $R_{\min}$ and $\bar{R}$ are computed over the six in-merge tasks only. Bold marks the best worst-task recovery for each omitted-task configuration.

Calibrated merging draws on available task information but does not reliably create missing task behaviour. ASR is substantially supported by the remaining adapters, while KWS, SV, and LID receive partial support; VS is essentially uncovered, and ER and IC each require an explicit adapter. For merging from fixed source adapters, Layer-wise is the recommended method when worst-task recovery matters, with the SV-omitted setting as the principal identified exception.

Chapter 7 tests an alternative that is not constrained by the source adapter span. Scalar and Layer-wise can only combine information already present in the stored task vectors, whereas the matched MTL baseline trains shared LoRA parameters on the same task set with access to pooled data. Whether that pooled training closes ER’s persistent gap and shifts the overall worst-task balance is the central question for the next chapter.

Chapter 7

Fixed-Suite MTL Comparison

Given independently trained adapters, can post-hoc coefficient calibration match a system trained jointly from the start? This chapter compares Scalar and Layer-wise merging with matched multi-task learning (MTL) to address that question. Scalar and Layer-wise reuse existing task vectors by estimating post-hoc composition coefficients; MTL instead retrains shared LoRA parameters directly on the pooled seven-task mixture. The distinction is practical as well as algorithmic: calibrated merging assumes source adapters already exist, whereas MTL requires all task data to be available for joint retraining. The backbone, LoRA capacity, target modules, prompts, and evaluation protocol are fixed throughout, so the comparison tests what post-hoc calibration can achieve against joint retraining in a matched configuration, not the ceiling of either method.

Throughout the chapter, MTL gives the highest non-ASR mean recovery; Layer-wise gives the strongest worst-task balance among calibrated methods while keeping ASR WER low; Scalar is the simplest reuse path with the lowest raw ASR WER. The chapter focuses on Scalar, Layer-wise, and MTL; Chapter 6 shows that Uniform, TIES, and DARE do not match these on global calibration or task balance. The fixed-suite comparison covers Base, Single-task, Scalar, Layer-wise, and MTL, all evaluated on the held-out test split.

7.1 Absolute Task Performance

Table 7.1 shows the fixed-suite results. For most non-ASR tasks MTL is strongest; ASR is the exception. MTL is highest on ER, IC, LID, SV, and VS, matching or slightly exceeding the Single-task reference on every task except SV, where it is marginally below (92.0 vs. 92.6); KWS is effectively tied, with Scalar only 0.1 pp above Layer-wise and MTL.

Model	ASR (WER ↓)	ER (F1 ↑)	IC (Acc ↑)	KWS (F1 ↑)	LID (Acc ↑)	SV (Acc ↑)	VS (Acc ↑)
Base	2.35	31.9	54.0	88.8	88.4	50.9	85.0
Single-task	2.02	45.7	85.5	92.2	99.9	92.6	94.5
Scalar	1.72	37.1	80.1	92.2	98.9	86.0	92.6
Layer-wise	1.81	42.6	83.9	92.1	98.6	89.8	93.8
MTL	2.24	46.1	85.8	92.1	100.0	92.0	94.5

Table 7.1: Seven-task absolute metrics in the matched MTL comparison. MTL leads most non-ASR tasks; both calibrated merges give lower ASR WER. Bold marks the best value among Scalar, Layer-wise, and MTL for each task.

Pooled-data shared-adapter training increases WER relative to both calibrated merges and the per-task ASR adapter (2.24% vs. 1.72%/1.81%/2.02%), consistent with gradient pressure from the six non-ASR tasks competing with the transcription objective during joint training. ER is the strongest contrast: Scalar remains below the Single-task reference (F1 37.1% vs. 45.7%), while MTL matches it at F1 46.1%. The main split is therefore visible before normalisation: MTL is strongest on most non-ASR tasks, while calibrated merging keeps ASR WER lower.

7.2 Recovery and Balance

Table 7.2 converts the same comparison into per-task recovery $R_i(m)$ (Equation 3.9), full mean $\bar{R}$, non-ASR mean, and worst-task recovery $R_{\min}$.

Method	ASR	ER	IC	KWS	LID	SV	VS	$\bar{R}$	$\bar{R}_{\text{ASR}}$	$R_{\min}$
Scalar	1.936	0.375	0.829	0.994	0.911	0.842	0.805	0.956	0.793	0.375
Layer-wise	1.652	0.773	0.947	0.962	0.883	0.934	0.929	1.011	0.905	0.773
MTL	0.338	1.029	1.009	0.948	1.006	0.986	1.003	0.903	0.997	0.338

Table 7.2: Seven-task matched comparison in headroom-normalised recovery. MTL leads on non-ASR mean; Layer-wise gives the strongest worst-task recovery. $\bar{R}_{\text{ASR}}$ excludes ASR from the mean. Bold marks the best value among Scalar, Layer-wise, and MTL for each column.

No method dominates all axes. MTL has the highest non-ASR mean recovery ($\bar{R}_{\text{ASR}} = 0.997$), but its worst-task score is set by ASR at $R = 0.338$. Layer-wise, however, gives the strongest worst-task recovery ($R_{\min} = 0.773$), ahead of Scalar (0.375) and MTL (0.338). Both calibrated merges fall below one on non-ASR mean recovery (0.905 and 0.793 respectively), reflecting ER’s persistent weakness. Pooled-data MTL is strongest on non-ASR mean, while calibrated merging gives a better ASR–balance trade-off in this configuration.

7.3 Held-Out Diagnostic Checks

Two held-out diagnostics check the fixed-suite conclusions outside the main recovery aggregates. SQA tests preservation of a related prompt-and-answer interface; LibriSpeech `test-other` tests the ASR pattern on a harder split. Neither changes the optimisation target: SQA is not a trained source task, merge-coefficient selection input, or MTL training task, and `test-other` is not used for ASR model selection.

7.3.1 SQA Format Stability

Table 7.3 gives the held-out MMSU SQA result. The adapted systems do not collapse the SQA interface: Scalar, Layer-wise, and MTL all sit above the frozen backbone, at 62.7%, 62.5%, and 63.5% versus 61.9%. This supports format stability for systems adapted to ASR, ER, IC, KWS, LID, SV, and VS, without treating SQA as an additional optimisation task.

	Base	Scalar	Layer-wise	MTL
Accuracy (%) $\uparrow$	61.9	62.7	62.5	63.5

Table 7.3: Held-out MMSU SQA overall accuracy. SQA is not a trained source task and is excluded from recovery aggregates.

The spread is too small for ranking the adapted systems to be meaningful. The supported conclusion is that adaptation to the seven source tasks preserves the held-out SQA prompt-and-answer format and overall accuracy, rather than demonstrating broad out-of-distribution generalisation. Appendix A.2 gives the category-level breakdown, where effects are mixed across MMSU subcategories.

7.3.2 Harder-Split ASR Check

Table 7.4 gives the LibriSpeech `test-other` check. This split is more difficult than `test-clean`, and it exposes the ASR trade-off directly in raw WER: Scalar and Layer-wise reduce WER below both the frozen backbone and the ASR single-task adapter, reaching 3.87% and 4.00% from 4.81% Base and 4.65% Single-task. MTL moves in the opposite direction, rising above the frozen backbone to 5.74% WER.

	Base	Single-task	Scalar	Layer-wise	MTL
WER (%) $\downarrow$	4.81	4.65	3.87	4.00	5.74

Table 7.4: LibriSpeech `test-other` WER. Scalar and Layer-wise reduce WER below the frozen backbone; MTL rises above it. Appendix results include Uniform.

This result is not a recovery-ratio artefact. On `test-clean`, MTL is already weaker than the calibrated merges for ASR; on `test-other`, the same gap appears in raw WER, with MTL rising above the frozen backbone. The ASR exception is therefore substantive: pooled-data MTL is strongest for non-ASR mean recovery, but calibrated merges give lower WER on both splits.

7.4 Six-Task Sensitivity

The six-task sensitivity check rules out an overly simple “MTL wins except for ASR” reading. The leave-one-out subsets from Section 6.3 show that MTL’s advantage depends on which task is omitted, not only on whether ASR is present. Across the seven six-task subsets, MTL’s $R_{\min}$ falls below both merge methods in six cases and turns negative in three: IC-omitted (-0.139), LID-omitted (-0.031), and SV-omitted (-0.503). Table 7.5 gives the full per-subset breakdown.

The omissions also clarify where the methods differ. Omitting IC or SV weakens ER and KWS under MTL specifically, suggesting that MTL’s joint optimisation exploits cross-task signal from those tasks in a way that fixed-coefficient merging does not. Calibrated merging

instead draws on individual task vectors and is less sensitive to any single omission. When ASR is omitted from the Layer-wise merge, the remaining six adapters still recover it at $R = 0.657$ (WER $\approx 2.13\%$; Table 6.5), suggesting that non-ASR task vectors encode ASR-supportive directions, a transfer effect that shared-adapter training does not replicate. Layer-wise gives the best worst-task recovery in four of seven subsets and remains the strongest full-suite calibrated method, but its leave-one-out advantage is not uniform.

Omitted	Scalar $R_{\min}$	Layer-wise $R_{\min}$	MTL $R_{\min}$	MTL $\bar{R}_{\text{ASR}}$
ASR	0.461	0.635	0.814	0.958
ER	0.840	0.871	0.253	0.997
IC	0.220	0.214	-0.139	0.808
KWS	0.538	0.717	0.219	0.998
LID	0.465	0.490	-0.031	0.985
SV	0.402	0.289	-0.503	0.874
VS	0.670	0.661	0.321	1.002

Table 7.5: Six-task subset comparison between calibrated merging and matched MTL. Each row omits one task; $R_{\min}$ is computed over the six included tasks only. Bold marks the best worst-task recovery among Scalar, Layer-wise, and MTL for each subset.

7.5 Practical Interpretation

The practical recommendation depends on which constraint matters most in the deployment setting. With a fixed task suite, pooled data, and feasible joint retraining, MTL is the stronger method for non-ASR mean recovery in this matched comparison, achieving $\bar{R}_{\text{excl. ASR}} = 0.997$. When adapters already exist, data access changes, ASR WER matters, or worst-task balance sets the deployment risk, calibrated merging is more relevant.

Within calibrated merging, Layer-wise gives the strongest worst-task balance, with the highest mean recovery ($\bar{R} = 1.011$) and the highest worst-task recovery ($R_{\min} = 0.773$) among all evaluated methods. Scalar is the simplest calibrated reuse method: one global coefficient recovers substantial performance and is more interpretable than layer-resolved calibration, at the cost of weaker task balance. Both calibrated merges give lower raw ASR WER than MTL (1.72% Scalar, 1.81% Layer-wise, vs. 2.24%), including on `test-other`.

Once adapters exist, recalibration uses substantially less additional data and compute than retraining MTL. Layer-wise coefficient estimation requires 14,400 examples and 5.65 hours, versus 230,228 examples and approximately 19.4 hours for matched MTL. This does not make merging data-free, remove adapter-set storage cost, or produce a compressed same-rank LoRA adapter. In this setup, calibrated merging is the lower additional-cost reuse path when task balance, ASR WER, or changing data access matters more than non-ASR mean recovery alone. Both comparisons assume a fixed task suite. Chapter 8 relaxes this assumption, asking whether layer-wise calibration remains viable as tasks arrive one at a time and whether the fixed-adapter reuse advantage persists as the composition grows.

Chapter 8

Continual Adapter Merging

The central question is whether layer-wise gradient-optimised merging enables balanced task addition over time. Five diagnostic task orders are evaluated, with two representative paths analysed in the main body and the full evidence summarised in Section 8.4. The criterion is acquisition–retention balance: acquisition measures incoming-task recovery at each stage, and retention measures the minimum recovery over tasks already present. Full stage-wise figures appear in Appendix A.5.

8.1 Setup and Success Criteria

Two methods extend the continual merge formulation in Equation 3.16, differing in how much flexibility they allow across layers. Two-scalar applies two global parameters $\alpha \geq 0$ and $\lambda \in [0, 1]$ uniformly at every layer. Layer-wise continual replaces these with per-layer parameters $(\alpha_\ell, \lambda_\ell)$, allowing each layer to weight the previous composition and the incoming adapter separately. Both use the gradient-based coefficient optimiser and checkpoint-selection framework from Section 4.4. Compressing previous tasks into a single composition $\tau^{(k)}$ means that a later merge can only balance the accumulated composition against the incoming adapter; it cannot selectively rebalance individual earlier task contributions without re-running the merge sequence from scratch using the individual source adapters.

Sequential MTL is included as a baseline. It continues training shared LoRA parameters as tasks arrive, re-training on all tasks seen so far at each stage, so retention depends on joint re-optimisation over accumulated task data rather than coefficient selection over fixed adapters. A balanced checkpoint is selected across all seen tasks.

Table 8.1 defines the five diagnostic paths. They cover both helping and conflicting task relationships from Chapter 5, not the full order space, and the main body analyses one path favourable to Sequential MTL and one to fixed-adapter Layer-wise continual merging.

Success is measured on two dimensions: acquisition (recovery of the newly added task at the current stage) and retention (the retained-task minimum, the weakest recovery over tasks already present). Mean recovery and per-task metrics are secondary checks. ASR WER is reported alongside recovery wherever ASR appears, because ASR’s narrow base-to-adapter headroom can amplify small improvements in the recovery metric.

Path	Addition order	Stress-test target
Acoustic-first	SV $\rightarrow$ LID $\rightarrow$ KWS $\rightarrow$ ASR	Acoustic grouping; ASR-final
Format-shift	ASR $\rightarrow$ IC $\rightarrow$ ER $\rightarrow$ SV	ASR-anchor retention; IC $\rightarrow$ ASR
ASR-held-out	SV $\rightarrow$ LID $\rightarrow$ KWS $\rightarrow$ VS	ASR transfer; acoustic retention
Semantic-first	IC $\rightarrow$ ER $\rightarrow$ ASR $\rightarrow$ VS $\rightarrow$ SV	Semantic-to-speaker; ER and ASR
Speaker-first	SV $\rightarrow$ VS $\rightarrow$ ASR $\rightarrow$ ER $\rightarrow$ IC	Order sensitivity vs. Semantic-first

Table 8.1: Continual diagnostic paths. Semantic-first and Speaker-first share the same five tasks in reverse order to probe order sensitivity; the set is targeted rather than exhaustive.

8.2 ASR-Held-Out Path

The ASR-held-out path accumulates SV, LID, and KWS, then adds vocal sound classification (VS) as the final task. ASR is evaluated throughout as an external probe but is excluded from the trained sequence, so none of the methods is required to retain an earlier ASR task.

Acquisition is similar across all three methods, but retained-task performance diverges. Sequential MTL leads on new-task acquisition, reaching $R = 1.006$ on VS with retained-task minimum $R = 0.857$ on KWS. Layer-wise continual acquires VS at $R = 0.950$ but achieves the best retained-task minimum of the three ($R = 0.894$ on SV). Two-scalar matches Layer-wise on acquisition ($R = 0.950$) while its retained-task minimum falls further, to $R = 0.652$ on SV.

This path identifies a favourable condition for Sequential MTL: when ASR is not a retained task and adapter reuse is not required, joint re-optimisation achieves the strongest acquisition–retention operating point. The contrast with the Acoustic-first path shows why this condition is order-dependent.

8.3 Acoustic-First Path

The Acoustic-first path accumulates SV, LID, and KWS, then adds ASR at the final stage. This tests whether continual merging can incorporate an incoming ASR adapter after a growing acoustic composition.

The methods diverge sharply on this path. Layer-wise continual is the only method that acquires ASR well, reaching $R = 1.657$ and WER = 1.81%, with retained-task minimum $R = 0.847$ on SV. Two-scalar under-acquires ASR ($R = 0.727$, WER = 2.11%) and its retained-task minimum also falls, to $R = 0.554$ on SV. Sequential MTL, by contrast, preserves the earlier acoustic tasks most strongly, with retained-task minimum $R = 0.916$ on KWS, but fails to acquire ASR at the final stage ($R = 0.151$, WER = 2.30%).

This path shows the complementary case for fixed-adapter reuse. Per-layer coefficients allow Layer-wise continual to rebalance the incoming ASR adapter against the accumulated composition at each layer. A single global trade-off, as in Two-scalar, is less flexible; Sequential MTL prioritises the retained acoustic objectives when ASR arrives last.

Table 8.2 compares the final-stage operating points for both representative paths.

Path	Method	Acq. (R)	Ret. min (R)	ASR WER (%)
ASR-held-out final task: VS	Layer-wise continual	0.950	0.894	1.90
	Two-scalar	0.950	0.652	1.89
	Sequential MTL	1.006	0.857	1.94
Acoustic-first final task: ASR	Layer-wise continual	1.657	0.847	1.81
	Two-scalar	0.727	0.554	2.11
	Sequential MTL	0.151	0.916	2.30

Table 8.2: Final-stage acquisition–retention for the two representative paths. In ASR-held-out, ASR WER is an external probe; in Acoustic-first, it is the direct acquisition metric.

8.4 Acquisition–Retention Balance Across Five Paths

Layer-wise continual gives the most reliable acquisition–retention balance across the five diagnostic paths: it is the only method that keeps the retained-task minimum at or above 0.60 in every path. The full five-path operating-point comparison and stage-wise trajectories are reported in Appendix A.5 (Table A.4 and Figure A.2).

By contrast, Two-scalar can acquire the incoming task competitively, but its single global balance between the previous composition and the incoming adapter leaves it more exposed to retained-task failures. In Semantic-first, ER falls to $R = 0.222$ under Two-scalar but remains at $R = 0.600$ under Layer-wise. In Speaker-first, Two-scalar reaches higher final-task acquisition than Layer-wise (0.937 versus 0.868), but this gain leaves SV at $R = 0.445$. These cases show the cost of using a single global parameter to balance acquisition and retention.

Sequential MTL’s ASR weakness is not explained by ASR’s position alone. Its retained ASR recovery falls below zero when ASR is an earlier retained task: Format-shift, Semantic-first, and Speaker-first give -0.396 , -0.126 , and -0.420 , with WERs of 2.48%, 2.39%, and 2.48%. It also acquires ASR poorly when ASR arrives last in Acoustic-first ($R = 0.151$). The favourable case is ASR-held-out, where ASR is only an external probe and never part of the optimisation mixture, indicating that the difficulty is tied to including ASR in Sequential MTL’s multi-task optimisation. For Layer-wise, large ASR recovery values in the retained ASR paths should again be read with raw WER (Section 6.2.2).

Stage-wise trajectories reveal two further patterns. Layer-wise can recover from temporary retained-task dips, as coefficients are reoptimised at each stage. Two-scalar can instead degrade a retained task progressively: in Semantic-first, its retained-task minimum falls from 0.859 at stage 2 to 0.222 at the final stage, whereas Layer-wise follows the same sequence from 0.843 to 0.600.

Together, these patterns make the practical choice depend on whether the task set will keep changing, adapters already exist, ASR is retained or incoming, and retraining on accumulated data is acceptable. Mean recovery alone does not confirm reliability: a poorly acquired incoming task or a single failed retained task can be hidden in the mean.

Chapter 9

Discussion and Conclusions

Independently trained speech-task LoRA adapters are reusable updates, not self-contained behavioural modules. Each adapter learns a useful change to the shared decoder, and that change can be stored, moved, and combined with others. What it does not carry is a guarantee about how it will behave once placed beside adapters it was never trained with. Reuse becomes dependable only after the adapters have been measured against one another and their contributions calibrated. Therefore, the project recasts modular speech adaptation as a problem of measurement and control, rather than only of where the weights are stored.

Behavioural modularity is not a property an adapter possesses in isolation: it acts through a shared decoder, so its effect on other tasks depends on what other adapters are present and what those tasks expect. A model that scores well on its own task can still shift the decoder’s behaviour on tasks it never saw, and these shifts are directional: one adapter may degrade a task that leaves it largely untouched in return. Task overfitting is the harmful end of this spectrum. Single-task evaluation can confirm source-task competence, but it does not establish deployable compatibility with the rest of the adapter set. Any reuse plan must begin by measuring cross-task effects on the adapters that will actually be composed.

Given these interactions, merging is better understood as calibration than as simple adapter addition. Averaging the task vectors recovers only about half of the single-task gains, at a mean recovery of 0.551, and the conflict-aware variants do not close that gap. The information a good merge needs is already present in the task-vector directions; the combined update is mainly mis-scaled. A single global coefficient chosen on development data raises mean recovery to 0.956, placing most of the failure in magnitude rather than in basic incompatibility. Allowing that scale to vary by layer gives the best balance in this suite: layer-wise calibration reaches a mean recovery of 1.011 while raising the weakest task to 0.773, with the source updates left untouched. Therefore, the reusable content of an adapter lies in its direction, while development data calibrate how strongly each adapter contributes and where that contribution is applied across the network.

Whether to reuse adapters in this way or retrain jointly depends on the deployment setting. The comparison with matched MTL draws that boundary clearly. When the task suite is fixed, the data can be pooled, and a fresh joint run is affordable, MTL has the easier problem and the highest non-ASR mean recovery, 0.997, because it optimises the shared parameters directly against the full mixture. Its advantage is not uniform across all objectives. MTL records higher ASR word error rate than either calibrated merge,

2.24% against 1.72% and 1.81%, and its weakest task can fall further than a layer-wise merge allows. Calibrated merging is the more relevant choice when adapters already exist, when requirements change and the suite is rebuilt from parts, when source data cannot be pooled, when ASR error rate governs the trade-off, or when the weakest task sets the deployment risk. In this setting, merging is valuable under those constraints, not a universal replacement for MTL.

The continual experiments test the same boundary under changing requirements. As the suite grows one task at a time, every method becomes order-dependent, so a task can be easy or hard to absorb according to when it arrives. Layer-wise continual merging gives the best balance between acquiring the new task and retaining the earlier ones, because it retunes the incoming adapter against the accumulated composition at each layer. This is the static calibration principle in a moving setting: static calibration balances a fixed adapter set, while continual calibration rebalances an existing composition whenever a new part is added. One diagnostic carries across both settings. Mean recovery can hide a single failed task, so worst-task recovery, not the average, decides whether a merge is safe to deploy.

The same point also gives the main bias and deployment implication of the project. Adapter composition can shift behaviour without changing the user-facing task interface: failures may appear as invalid formats, systematically wrong labels, or concentrated degradation on one task, class, language, or speaker-verification condition. A new composition should therefore not be approved for deployment on the basis of aggregate recovery alone; it needs task-level validation, including output validity, per-class behaviour for classification tasks, and worst-task recovery.

These conclusions are bounded by their setting. They rest on one backbone and model family, a single LoRA configuration, this task suite, the MTL recipes tested here, and a small set of diagnostic task orders. The leave-one-out results are a standing reminder that merging can only rebalance the source vectors it holds and will not synthesise a missing adapter. The most useful next step is to predict compatibility before paying for calibration. At present, risky task pairs and scale-sensitive combinations surface only through cross-task evaluation and development-set tuning, while plain norm and cosine measures track the real interference poorly. A predictor that flagged these cases in advance would remove much of that cost, although it would have to reason about more than global weight-space geometry. Beyond prediction, the open question is reach: whether the same calibration behaviour holds across other speech-language backbones, task mixtures, and adapter configurations.

LoRA made speech-task adaptation modular in storage; calibrated merging makes its reuse accountable. With cross-task measurement to expose the interactions, calibrated reuse to control them, and worst-task monitoring to bound risk, a set of independently trained adapters can be assembled into a working multi-task system and reassembled as requirements change. The adapters are genuinely reusable. They are not, and should not be treated as, plug-and-play behavioural modules.

Bibliography

- [1] Changli Tang, Wenyi Yu, Guangzhi Sun, et al. SALMONN: Towards generic hearing abilities for large language models. In *ICLR*, 2024.
- [2] Yunfei Chu, Jin Xu, Qian Yang, et al. Qwen2-Audio technical report. *arXiv preprint arXiv:2407.10759*, 2024.
- [3] Jin Xu, Zhifang Guo, Jinzheng He, et al. Qwen2.5-Omni technical report. *arXiv preprint arXiv:2503.20215*, 2025.
- [4] Dong Zhang, Shimin Li, Xin Zhang, et al. SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities. In *Findings of ACL: EMNLP*, pages 15757–15773, 2023.
- [5] Yuan Gong, Hongyin Luo, Alexander H. Liu, et al. Listen, think, and understand. In *ICLR*, 2024.
- [6] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- [7] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *ICML*, 2022.
- [8] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, et al. Editing models with task arithmetic. In *ICLR*, 2023.
- [9] Jinghan Zhang, Shiqi Chen, Junteng Liu, et al. Composing parameter-efficient modules with arithmetic operations. In *NeurIPS*, 2023.
- [10] Prateek Yadav, Derek Tam, Leshem Choshen, et al. TIES-merging: Resolving interference when merging models. In *NeurIPS*, 2023.
- [11] Le Yu, Bowen Yu, Haiyang Yu, et al. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *ICML*, 2024.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *NeurIPS*, 2017.
- [13] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, et al. Parameter-efficient transfer learning for NLP. In *ICML*, pages 2790–2799, 2019.
- [14] Vassil Panayotov, Guoguo Chen, Daniel Povey, et al. LibriSpeech: An ASR corpus based on public domain audio books. In *ICASSP*, pages 5206–5210, 2015.
- [15] Emanuele Bastianelli, Andrea Vanzo, Pawel Swietojanski, et al. SLURP: A spoken language understanding resource package. In *EMNLP*, pages 7252–7262, 2020.
- [16] Pete Warden. Speech commands: A dataset for limited-vocabulary speech recognition. *arXiv preprint arXiv:1804.03209*, 2018.
- [17] Soujanya Poria, Devamanyu Hazarika, Navonil Majumder, et al. MELD: A multimodal multi-party dataset for emotion recognition in conversations. In *ACL*, pages 527–536, 2019.
- [18] Yuan Gong, Jin Yu, and James Glass. VocalSound: A dataset for improving human vocal sounds recognition. In *ICASSP*, pages 151–155, 2022.
- [19] Joon Son Chung, Arsha Nagrani, and Andrew Senior. VoxCeleb2: Deep speaker recognition. In *Interspeech*, pages 1086–1090, 2018.
- [20] Alexis Conneau, Min Ma, Simran Khanuja, et al. FLEURS: Few-shot learning evaluation of universal representations of speech. In *IEEE SLT*, pages 798–805, 2023.
- [21] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic

- forgetting in neural networks. *PNAS*, 114(13):3521–3526, 2017.
- [22] Enneng Yang, Li Shen, Guibing Guo, et al. Model merging in LLMs, MLLMs, and beyond: Methods, theories, applications and opportunities. *ACM Comput. Surv.*, 58(8):216:1–216:41, 2026.
 - [23] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, et al. Loss surfaces, mode connectivity, and fast ensembling of DNNs. In *NeurIPS*, 2018.
 - [24] Enneng Yang, Zhenyi Wang, Li Shen, et al. AdaMerging: Adaptive model merging for multi-task learning. In *ICLR*, 2024.
 - [25] Haoyu Yang, Zheng Zhang, and Saket Sathe. SuperMerge: An approach for gradient-based model merging. *arXiv preprint arXiv:2412.10416*, 2024.
 - [26] Chengsong Huang, Qian Liu, Bill Yuchen Lin, et al. LoraHub: Efficient cross-task generalization via dynamic LoRA composition. In *COLM*, 2024.
 - [27] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, et al. AdapterFusion: Non-destructive task composition for transfer learning. In *EACL*, pages 487–503, 2021.
 - [28] Michael S. Matena and Colin A. Raffel. Merging models with fisher-weighted averaging. In *NeurIPS*, 2022.
 - [29] Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, et al. Dataless knowledge fusion by merging weights of language models. In *ICLR*, 2023.
 - [30] Enneng Yang, Li Shen, Zhenyi Wang, et al. Representation surgery for multi-task model merging. In *ICML*, 2024.
 - [31] Derek Tam, Mohit Bansal, and Colin Raffel. Merging by matching models in task parameter subspaces. *TMLR*, 2024.
 - [32] George Stoica, Pratik Ramesh, Boglarka Ecsedi, et al. Model merging with SVD to tie the KnOTS. In *ICLR*, 2025.
 - [33] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.
 - [34] Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. In *NeurIPS*, 2018.
 - [35] Trevor Standley, Amir R. Zamir, Dawn Chen, et al. Which tasks should be learned together in multi-task learning? In *ICML*, 2020.
 - [36] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, et al. Gradient surgery for multi-task learning. In *NeurIPS*, 2020.
 - [37] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, et al. GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *ICML*, 2018.
 - [38] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *CVPR*, 2018.
 - [39] David Rolnick, Arun Ahuja, Jonathan Schwarz, et al. Experience replay for continual learning. In *NeurIPS*, 2019.
 - [40] Anke Tang, Enneng Yang, Li Shen, et al. Merging models on the fly without retraining: A sequential approach to scalable continual model merging. *arXiv preprint arXiv:2501.09522*, 2025.
 - [41] Yi-Lin Sung, Linjie Li, Kevin Lin, et al. An empirical study of multimodal model merging. In *Findings of ACL: EMNLP*, pages 1563–1575, 2023.
 - [42] Anirudh S. Sundar, Chao-Han Huck Yang, David M. Chan, et al. Multimodal attention merging for improved speech recognition and audio event classification. In *ICASSP Workshop on Self-supervision in Audio, Speech and Beyond*, 2024.
 - [43] Dingdong Wang, Junan Li, Jincenzi Wu, et al. MMSU: A massive multi-task spoken language understanding and reasoning benchmark. In *ICLR*, 2026.

Appendix A

Supplementary Diagnostics

This appendix collects targeted diagnostics and full supporting sweeps for the experimental chapters. The checks are secondary evidence: they clarify parsing, held-out behaviour, ASR headroom, and continual path dependence without replacing the main results.

A.1 Output-Format Validity

This diagnostic separates parser failures from valid-but-wrong label predictions for the classification drops in Chapter 5. Strict label parsing can inflate apparent drops when generated text fails to match any known label, so the invalid-output rate checks whether the Chapter 5 failures are parser artefacts or valid-label errors. Table A.1 records invalid-output rates across methods and classifiable tasks.

Task	Base	Single-task	Scalar	Layer-wise	MTL
ER	0.00	0.00	0.08	0.00	0.00
IC	2.28	0.04	1.12	0.07	0.00
KWS	1.03	0.00	0.27	0.20	0.00
LID	0.07	0.01	0.00	0.00	0.00
SV	0.00	0.00	0.00	0.00	0.00
VS	0.00	0.00	0.00	0.00	0.00

Table A.1: Invalid-output rates on the test split, by task and method. Values are percentages of evaluated examples; ASR is excluded because it uses WER rather than label matching. Except for IC under Base and Scalar, invalid-output rates are at or below 0.27%, so the main classification failures are generally valid-but-wrong labels rather than parser artefacts.

The exception is IC under Base (2.28%) and Scalar (1.12%), where invalidity is noticeable. However, the primary off-task failures discussed in Chapter 5 are IC→LID and KWS→LID, which have near-perfect label recognition rates; the IC invalidity does not explain those failures.

A.2 Held-Out SQA Diagnostic

Chapter 7 reports the overall held-out SQA headline. This appendix gives category-level detail for the same diagnostic. SQA is not a trained source task: it provides no training signal, no coefficient-selection input, and is excluded from all recovery aggregates. Table A.2 records category-level results as a held-out stability check, not as evidence for broad out-of-distribution generalisation.

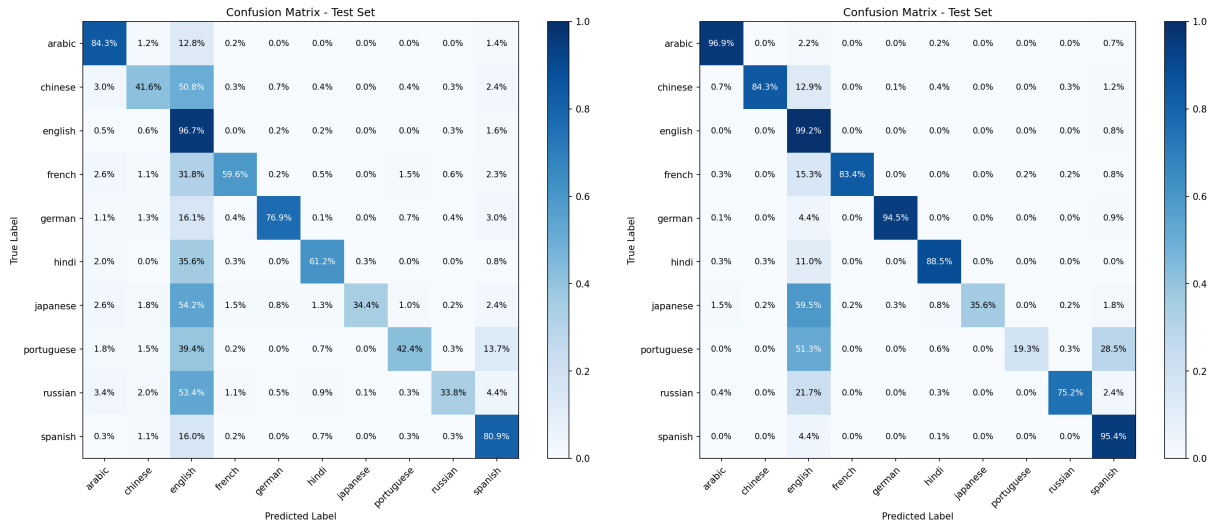
Model	Overall acc.	Reason.	Percept.	Detect.	Recogn.	Compar.	Lang.
Base	61.9	80.5	37.6	63.6	59.1	26.9	85.8
Scalar merge	62.7	79.9	41.6	64.2	59.3	24.8	85.9
Layer-wise merge	62.5	79.4	42.1	63.8	59.2	23.2	86.4
MTL	63.5	80.6	41.2	66.4	61.3	25.3	87.2

Table A.2: Held-out MMSU SQA diagnostic. Values are percentages; category columns report macro accuracy over MMSU subtasks for Reasoning, Perception, Detection, Recognition, Comparison, and Language.

Category effects are mixed: Perception improves across all adapted systems and Language is stable or slightly higher, while Comparison is consistently below Base. The results support limited stability, not a broad out-of-distribution generalisation claim.

A.3 LID Confusion Diagnostics

This diagnostic examines the output-side failure mode underlying the IC→LID and KWS→LID results identified in Chapter 5. The confusion matrices distinguish parseable label-prior shifts from invalid response formats.



(a) IC adapter evaluated on LID. The main failure mode is over-prediction of English for several non-English classes, especially Japanese, Russian, and Chinese.

(b) KWS adapter evaluated on LID. The strongest visible error is Japanese utterances being assigned to English, consistent with shifted label priors rather than parser failure.

Figure A.1: LID confusion matrices for the largest parseable off-task classification failures in Chapter 5. Rows are true labels and columns are predicted labels. Both failures show over-prediction of English for several non-English utterances, consistent with shifted label priors rather than parser failure.

Chapter 5 identifies IC→LID and KWS→LID as the largest valid-label classification failures. Figure A.1 makes the output-side evidence visible: both failures are dominated by parseable LID labels rather than unrecognised output, with a systematic shift towards English predictions for several non-English classes.

A.4 ASR Same-Sample WER Diagnostic

The narrow Base-to-Single-task WER headroom means recovery ratios can amplify small absolute changes. This diagnostic checks whether the ASR improvement in Chapter 6 is visible in raw error counts or is an artefact of that headroom. Table A.3 gives a same-sample check on the ASR-only scalar comparison: more utterances improve than worsen, and substitutions, insertions, and deletions all fall under the scaled update.

Diagnostic	Value	Interpretation
Comparison	ASR adapter vs. scaled ASR ($\gamma = 0.417$)	Same-sample dev-clean comparison under aggressive ASR normalisation.
WER	2.18→1.83	Scaling the ASR task vector reduces raw WER, not only recovery ratio.
Utterance counts	296 / 186 / 2221	More utterances improve than worsen; the final value is unchanged utterances.
Edit operations	Subs. 901→781; ins. 109→81; dels. 174→135	Substitutions, insertions, and deletions all fall under the scaled ASR update.

Table A.3: Same-sample ASR pairwise diagnostic on dev-clean. WER values are percentages under aggressive normalisation; the operation columns show total substitutions, insertions, and deletions before and after scaling the ASR task vector ($\gamma = 0.417$). All three error categories decrease under the scaled update, showing that the WER improvement is visible in raw error counts rather than only in the recovery ratio.

Archived ASR-containing merge checks are consistent with these raw WER results: all post-hoc merges in the seven-task table improve raw WER over the single-task ASR adapter, and a six-task same-sample check found lower WER for three ASR-containing merges (Uniform, Scalar, Layer-wise). These checks are included as supporting context only; ASR headroom remains small, so the result should not be read as a broad robustness claim.

A.5 Continual Path Sweep

This section gives the full five-path acquisition–retention sweep supporting Chapter 8. The paths are diagnostic, not exhaustive: they test selected orderings suggested by the interference patterns in Chapter 5. The main body analyses ASR-held-out and Acoustic-first; Figure A.2 shows the stage-wise trajectories, and Table A.4 records the final-stage operating points.

The three paths not analysed in the main body add the same qualification from different orderings. In Format-shift, Sequential MTL acquires the final SV task most strongly but pushes retained ASR below base, while the merge methods trade some acquisition for positive retained-task minima. In the Semantic-first and Speaker-first reversal pair, Two-scalar can acquire the final task more strongly than Layer-wise, but its weakest retained task falls much further. Layer-wise is therefore the only continual method whose final retained-task minimum stays at or above $R = 0.60$ in all five paths.

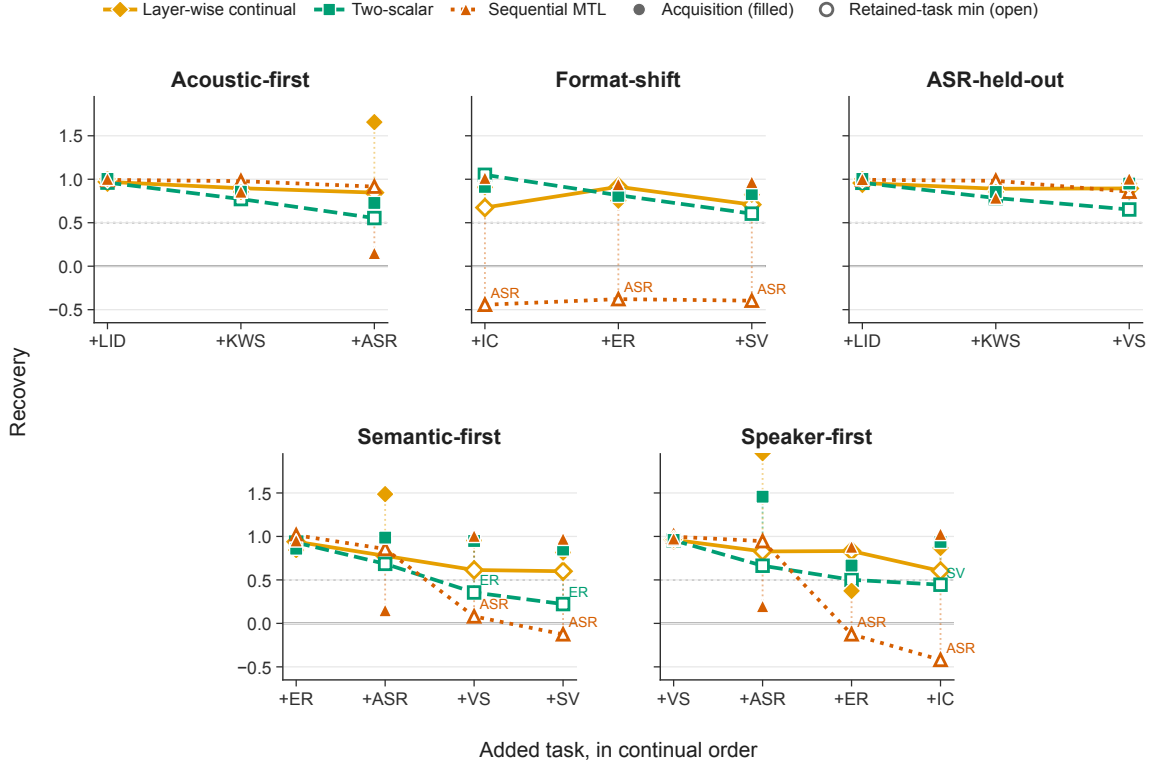


Figure A.2: Stage-wise acquisition and retention over continual additions for the five diagnostic paths. Filled markers trace acquisition of the newly added task and open markers trace retained-task minimum. Sub-0.5 labels mark the weakest retained task where a stage produces a retained-task failure.

Path	Method	Final task	Acq.	Weakest task	Ret. min
Acoustic-first	Layer-wise continual	ASR	1.657	SV	0.847
Acoustic-first	Two-scalar	ASR	0.727	SV	0.554
Acoustic-first	Sequential MTL	ASR	0.151	KWS	0.916
Format-shift	Layer-wise continual	SV	0.823	ER	0.708
Format-shift	Two-scalar	SV	0.830	ER	0.604
Format-shift	Sequential MTL	SV	0.968	ASR	-0.396
ASR-held-out	Layer-wise continual	VS	0.950	SV	0.894
ASR-held-out	Two-scalar	VS	0.950	SV	0.652
ASR-held-out	Sequential MTL	VS	1.006	KWS	0.857
Semantic-first	Layer-wise continual	SV	0.815	ER	0.600
Semantic-first	Two-scalar	SV	0.849	ER	0.222
Semantic-first	Sequential MTL	SV	0.972	ASR	-0.126
Speaker-first	Layer-wise continual	IC	0.868	ER	0.603
Speaker-first	Two-scalar	IC	0.937	SV	0.445
Speaker-first	Sequential MTL	IC	1.028	ASR	-0.420

Table A.4: Exact final-stage acquisition-retention operating points for the five continual diagnostic paths.

Appendix B

Risk Assessment Retrospective

The initial risk assessment identified this as a computer-based project, with extended computer use as the main practical hazard. That was broadly appropriate for the work performed: developing, training, evaluating, and merging Speech LLM LoRA adapters using a personal laptop and SSH access to a departmental workstation. The project did not involve laboratory work, mechanical equipment, electrical construction, fieldwork, or human-subject data collection by the author.

The hazards encountered were minimal and mainly ergonomic. The relevant controls were sensible workstation setup, posture, screen use, and regular breaks during long coding and writing sessions. No significant incidents or near misses occurred.

If starting again, I would still assess the physical risk as low, but I would state the computational and project-management controls more explicitly. In practice, the main risks were keeping a clear run log, preserving result artefacts, managing storage for large model outputs, and avoiding unnecessary repeated training on shared compute.